Oracy AI
Planning Documentation

Complete Technical Specification

Generated: 2026-03-03 13:56:51

MVP Implementation Plan

Page 1 of 349

Oracy AI - Implementation Plan

Executive Summary

Oracy AI is a pilot-ready MVP that produces teacher-verifiable, inspection-grade evidence of oracy progression from a single student recording (solo mode only in MVP). The system uses a multi-stage AI pipeline to analyze video/audio recordings and generate structured assessment reports across four strands: Physical, Linguistic, Cognitive, and Social-emotional.

Table of Contents

1. Architecture Overview

- System Architecture Diagram
- High-Level Data Flow
- Core Components

2. Tech Stack

Page 2 of 349

- Frontend (React)
- Backend (Django)
- Database & Storage
- AI/ML Services
- DevOps & Infrastructure

3. AWS Infrastructure

- VPC & Networking
- Compute (ECS/Fargate, EC2)
- Storage (S3, RDS, ElastiCache)
- AI/ML Services (AWS Transcribe, SageMaker)
- Security & IAM
- CDN & CloudFront

4. AI/ML Pipeline

- Speech-to-Text (OpenAI Whisper/AWS Transcribe)

Page 3 of 349

- Deterministic Feature Extraction
- Benchmarking Layer (Core IP)
- Evidence Candidate Generation
- Rubric-Constrained LLM Scoring
- Model Selection & Providers

5. Backend Structure

- Django Project Structure
- Apps & Responsibilities
- Database Schema
- API Design (REST + WebSocket)
- Background Jobs (Celery)

6. Frontend Structure

- React Project Structure
- Component Architecture

Page 4 of 349

- State Management
- Routing & Navigation
- Video Recording & Playback

7. Database Schema

- Core Tables
- Relationships
- Indexes & Constraints
- Data Retention

8. Security & Compliance

- Authentication & Authorization
- Data Protection (GDPR/Schrems II)
- Video Storage Security
- Audit Logging

Page 5 of 349

9. Development Roadmap

- Week 1 Deliverables
- Week 2 Deliverables
- Post-MVP Phases
- Future Model Training Strategy

10. Deployment Strategy

- CI/CD Pipeline
- Environment Management
- Blue/Green Deployment
- Monitoring & Alerting

Quick Reference

MVP Assessment Modes

Page 6 of 349

- **Presenting** - Solo presentation skills
- **Explaining** - Explanation and clarification
- **Persuading** - Argumentation and persuasion

Four Assessment Strands

1. **Physical** - Delivery, pace, volume, fillers
2. **Linguistic** - Vocabulary, clarity, register
3. **Cognitive** - Structure, reasoning, examples
4. **Social-emotional** - Intention, tone, audience awareness

Band Levels

- **Emerging** - Below age expectation
- **Expected** - At age expectation
- **Exceeding** - Above age expectation

AI Pipeline Stages

1. Speech-to-Text (Timestamped)
2. Deterministic Feature Extraction
3. Benchmarking Layer
4. Evidence Candidate Generation

Page 7 of 349

Document Version: 1.0
Last Updated: March 2026
Status: Draft

Architecture Overview

System Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                              CLIENT LAYER (React)                            │
├──────────────────────────────────────────────────────────────────────────────┤
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌────────────────────┐ │
│  │  Auth        │  │  Recording   │  │  Dashboard   │  │  Teacher Review UI  │ │
│  │  (Login)     │  │  (WebRTC)    │  │  (Reports)   │  │  (Score/Clip/Feedback) │ │
│  └──────┬───────┘  └──────┬───────┘  └──────┬───────┘  └──────────┬──────────┘ │
└─────────┼─────────────────┼─────────────────┼────────────────────┼────────────┘
          │                 │                 │                    │
          └─────────────────┴─────────────────┴────────────────────┘
                                          │ HTTPS/WSS
```


Page 8 of 349

```
                                ▼

┌──────────────────────────────────────────────────────────────────────────┐

│                        API GATEWAY (AWS ALB/CloudFront)                  │

├──────────────────────────────────────────────────────────────────────────┤

│                      SSL Termination / Rate Limiting                     │

└──────────────────────────────────────────────────────────────────────────┘

                                │

                                ▼

┌──────────────────────────────────────────────────────────────────────────┐

│                        BACKEND LAYER (Django)                            │

├──────────────────────────────────────────────────────────────────────────┤

│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────────────────┐ │

│  │  REST API    │  │  WebSocket   │  │  Background │  │  Admin Interface       │ │

│  │  (DRF)       │  │  (Channels) │  │  (Celery)    │  │  (Django Admin)        │ │

│  └──────┬───────┘  └──────┬───────┘  └──────┬───────┘  └──────────────────────┘ │

└─────────┼─────────────────┼─────────────────┼──────────────────────────────────┘

          │                 │                 │

          │                 │                 └──────────────────┐

          │                 │                                    │

          ▼                 ▼                                    ▼

┌──────────────────────────────────────────────────────────────────────────┐

│                        AI PIPELINE SERVICES                              │

├──────────────────────────────────────────────────────────────────────────┤
```


Page 9 of 349

```
│                                                                          │
│  ┌──────────────┐   ┌──────────────┐   ┌──────────────┐   ┌──────────────────┐ │
│  │  STT Service │──▶│  Feature     │──▶│  Benchmark   │──▶│  LLM Scoring     │ │
│  │  (OpenAI/    │   │  Extraction  │   │  Engine      │   │  (OpenAI/        │ │
│  │   AWS)       │   │  (Python)    │   │  (BDL)       │   │   Claude)        │ │
│  └──────────────┘   └──────────────┘   └──────────────┘   └──────────────────┘ │
│        │                                                          │   │
│        │        ┌──────────────────────────────────────────────────┘   │
│        │        │                                                       │
│        │        ▼                                                       │
│        │    ┌──────────────────┐                                        │
│        └───▶│  Evidence Gen    │                                        │
│             │  (Deterministic) │                                        │
│             └──────────────────┘                                        │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
         │
         ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                          DATA LAYER                                      │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  ┌──────────────┐   ┌──────────────┐   ┌──────────────┐   ┌──────────────────┐ │
```


Page 10 of 349

```
│  │  PostgreSQL  │   │  Redis       │   │  S3 Storage  │   │  Elasticsearch   │ │

│  │  (RDS)       │   │  (ElastiCache)│  │  (Video/Audio)│  │  (Optional:      │ │

│  │              │   │              │   │              │   │   Search/Logs)   │ │

│  └──────────────┘   └──────────────┘   └──────────────┘   └──────────────────┘ │

│                                                                                 │

└─────────────────────────────────────────────────────────────────────────────────┘
```

High-Level Data Flow

1. Recording Upload Flow

```
Student/Teacher → React App → Django API → S3 (Presigned URL)

                                        ↓

                                  Trigger Celery Job

                                        ↓

                              ┌──────────────────┐

                              │  AI Pipeline     │

                              │  - STT           │

                              │  - Features      │
```


Page 11 of 349

```
                              │  - Scoring      │

                              └─────────┬───────┘

                                        ↓

                              Store Results in DB

                                        ↓

                              WebSocket Notification

                                        ↓

                              Teacher Review Available
```

2. Assessment Processing Flow

```
┌─────────────────┐     ┌─────────────────┐     ┌─────────────────┐

│  Video Upload   │────▶│  Extract Audio  │────▶│  Speech-to-Text │

└─────────────────┘     └─────────────────┘     └─────────┬───────┘

                                                          │

                                                          ▼

┌─────────────────┐     ┌─────────────────┐     ┌─────────────────┐

│  LLM Scoring    │◀────│  Benchmarking   │◀────│  Feature        │

│  & Evidence     │     │  (BDL)          │     │  Extraction     │

└────────┬────────┘     └─────────────────┘     └─────────────────┘

         │
```


Page 12 of 349

```
        ▼

┌────────────────┐     ┌────────────────┐     ┌────────────────┐

│  Draft Report  │────▶│  Teacher Review│────▶│  Signed Report │

│  Generated     │     │  & Edit        │     │  (Canonical)   │

└────────────────┘     └────────────────┘     └────────────────┘
```

3. Teacher Review Flow

```
Teacher Opens ──▶  View AI Draft  ──▶  Edit Scores/  ──▶  Add Private  ──▶  Sign Off

Assessment         Report             Evidence/          Notes             (Audit Trai

                                       Feedback                            Captured)
```

Core Components

Frontend (React)

- **Authentication Module**: JWT-based auth with role-based access

Page 13 of 349

- **Recording Component**: WebRTC-based video/audio capture
- **Dashboard**: Assessment overview, analytics, cohort views
- **Review Interface**: Side-by-side video player with editing tools
- **PDF Export**: Client-side PDF generation for reports

Backend (Django)

- **API Layer**: Django REST Framework for all endpoints
- **WebSocket Layer**: Django Channels for real-time updates
- **Task Queue**: Celery + Redis for async AI processing
- **Admin Interface**: Built-in Django admin for data management
- **Core Apps**:
- `users`: Authentication and authorization
- `students`: Student profiles and cohort management
- `assessments`: Recording metadata and workflow
- `analysis`: AI pipeline orchestration
- `reports`: Draft and signed report management

Page 14 of 349

- `benchmarks` : BDL versioning and scoring logic

AI Pipeline

- **STT Service**: OpenAI Whisper API or AWS Transcribe
- **Feature Extractor**: Python library using librosa, NLTK, spaCy
- **Benchmark Engine**: JSON-based BDL interpreter
- **Evidence Generator**: Deterministic candidate selection
- **LLM Scorer**: OpenAI GPT-4o / Claude API with structured output

Infrastructure (AWS)

- **Compute**: ECS Fargate for containerized services
- **Database**: RDS PostgreSQL (Multi-AZ)
- **Cache**: ElastiCache Redis
- **Storage**: S3 with versioning and lifecycle policies
- **CDN**: CloudFront for static assets and video streaming

Page 15 of 349

- **Queue**: SQS or Redis for Celery backend
- **Monitoring**: CloudWatch, X-Ray

Technology Decisions

Why React + Django?

- **React**: Component reusability, strong ecosystem, excellent video/media handling
- **Django**: Rapid development, robust ORM, built-in admin, excellent Python ecosystem for ML
- **Separation**: Allows independent scaling of frontend and backend

Why AWS?

- **AWS Transcribe**: Purpose-built for speech-to-text with timestamps
- **S3**: Industry-standard object storage with fine-grained permissions
- **ECS Fargate**: Serverless containers - no server management

Page 16 of 349

- **RDS**: Managed PostgreSQL with automated backups

AI Model Strategy

- **MVP**: Use external APIs (OpenAI, Anthropic) - faster time-to-market
- **Post-MVP**: Train custom models using collected teacher feedback data
- **Hybrid**: Keep deterministic feature extraction separate from LLM scoring

Scalability Considerations

Horizontal Scaling

- Stateless Django backend - can run multiple containers
- Celery workers scale independently based on queue depth
- S3 handles unlimited video storage

Page 17 of 349

Performance Optimization

- Video upload uses presigned URLs (direct to S3)
- AI processing is fully asynchronous
- Response caching for benchmark definitions
- CDN for video streaming and static assets

Data Retention

- Video files: 7-year retention (educational records)
- Audit logs: Permanent
- AI processing artifacts: 1 year
- Soft deletes for all records (compliance)

Security Architecture

Data Protection

Page 18 of 349

- All data encrypted at rest (S3, RDS)
- TLS 1.3 for all communications
- Video access via time-limited presigned URLs only
- No public video URLs ever

Access Control

- Role-based: Admin, Teacher, Student, Parent (future)
- Row-level permissions (teachers only see their students)
- API key management for external services

Compliance

- GDPR-compliant data handling
- Audit trail for all modifications
- Right to erasure support

Page 19 of 349

- Data residency controls (future)

Tech Stack

Overview

Layer	Technology	Version/Notes
Frontend	React + TypeScript	v18+, Vite build
Backend	Django + Django REST Framework	v5.0+, Python 3.12
Database	PostgreSQL	15+
Cache	Redis	7+
Message Queue	Redis (Celery) or AWS SQS	-
AI/ML	OpenAI API, AWS Transcribe, Anthropic	Latest stable
Infrastructure	AWS (ECS, RDS, S3, CloudFront)	-
DevOps	Docker, GitHub Actions, Terraform	-

Frontend Stack (React)

Core Framework

Page 20 of 349

- **React 18+**: Modern React with concurrent features
- **TypeScript 5+**: Type safety throughout
- **Vite**: Fast development builds and optimized production bundles

State Management

- **Zustand**: Lightweight state management for global state
- **React Query (TanStack Query)**: Server state management, caching, synchronization
- **React Hook Form**: Form state management with validation

Routing

- **React Router v6**: Client-side routing with lazy loading

UI Component Library

Page 21 of 349

- **Tailwind CSS**: Utility-first CSS framework
- **Headless UI**: Unstyled, accessible UI primitives
- **Lucide React**: Icon library

Video & Media

- **MediaRecorder API**: Native browser recording
- **React Player**: Video playback component
- **wavesurfer.js**: Audio waveform visualization

HTTP Client

- **Axios**: HTTP requests with interceptors for auth

Authentication

Page 22 of 349

- **JWT (jsonwebtoken)**: Token-based authentication
- **react-jwt**: JWT decoding in React

Testing

- **Vitest**: Unit testing framework
- **React Testing Library**: Component testing
- **Playwright**: E2E testing

Build Tools

- **Vite**: Build tool and dev server
- **ESLint**: Code linting
- **Prettier**: Code formatting

Page 23 of 349

Backend Stack (Django)

Core Framework

- **Django 5.0+**: Web framework
- **Django REST Framework 3.15+**: REST API framework
- **Django Channels**: WebSocket support for real-time updates
- **Python 3.12**: Latest Python with performance improvements

Database & ORM

- **PostgreSQL 15+**: Primary database
- **psycopg3**: PostgreSQL adapter
- **django-migrations**: Schema versioning

Page 24 of 349

Background Tasks

- **Celery 5.3+**: Distributed task queue
- **Redis**: Celery message broker and result backend
- **django-celery-beat**: Periodic task scheduler

Authentication & Security

- **django-cors-headers**: CORS handling
- **PyJWT**: JWT token generation/validation
- **django-ratelimit**: API rate limiting
- **django-guardian**: Object-level permissions
- **argon2-cffi**: Secure password hashing

API & Documentation

- **drf-spectacular**: OpenAPI 3 schema generation

Page 25 of 349

- **django-filter**: Query parameter filtering

Media & Storage

- **django-storages**: Storage backends
- **boto3**: AWS SDK for Python (S3 interaction)

AI/ML Integration

- **openai**: OpenAI Python client
- **anthropic**: Anthropic Python client
- **boto3**: AWS SDK (Transcribe, etc.)

Audio Processing (Feature Extraction)

- **librosa**: Audio analysis (tempo, rhythm, spectral features)
- **pydub**: Audio manipulation

Page 26 of 349

- **soundfile**: Audio file I/O
- **numpy**: Numerical computing

Text Processing

- **spaCy**: NLP (sentence parsing, tokenization)
- **NLTK**: Natural language toolkit
- **textstat**: Readability metrics

Data Validation

- **pydantic**: Data validation using Python type hints
- **marshmallow**: Object serialization

Testing

Page 27 of 349

- **pytest**: Testing framework
- **pytest-django**: Django pytest integration
- **factory-boy**: Test data factories
- **freezegun**: Time manipulation in tests
- **responses**: Mock HTTP requests
- **coverage.py**: Code coverage

Monitoring & Logging

- **sentry-sdk**: Error tracking
- **structlog**: Structured logging
- **django-prometheus**: Metrics export

Database Stack

Page 28 of 349

Primary Database

- **PostgreSQL 15+** (AWS RDS)
- Multi-AZ deployment for high availability
- Automated backups (7-day retention)
- Encryption at rest (KMS)

Key PostgreSQL Extensions

- **uuid-ossp**: UUID generation
- **pg_trgm**: Trigram similarity (fuzzy search)
- **btree_gist**: GiST index operator

Cache

- **Redis 7+** (AWS ElastiCache)

Page 29 of 349

- Session storage
- Celery broker
- API response caching
- Real-time pub/sub (WebSocket)

Search (Future)

- **Elasticsearch 8+** or **OpenSearch**
- Full-text search for transcripts
- Assessment history search

AI/ML Services Stack

Speech-to-Text (STT)

Page 30 of 349

Service	Use Case	Pros	Cons
OpenAI Whisper API	Primary STT	High accuracy, timestamp support, cost-effective	External dependency
AWS Transcribe	Fallback/Alternative	AWS native, speaker diarization (future)	Higher cost
Whisper (self-hosted)	Future option	Full control, no external calls	Infrastructure overhead

Decision: Start with OpenAI Whisper API, abstract behind service interface for easy swapping.

Large Language Models (LLM)

Model	Use Case	Context Window	Cost
OpenAI GPT-4o	Primary scoring	128K	Medium
Anthropic Claude 3.5 Sonnet	Fallback/comparison	200K	Medium
OpenAI GPT-4o-mini	Lightweight tasks	128K	Low

Decision: GPT-4o for main scoring pipeline (structured output), Claude for evaluation/comparison.

Audio Feature Extraction

Library	Purpose
librosa	Tempo, rhythm, spectral features, pause detection
pydub	Audio slicing, format conversion, volume analysis
webrtcvad	Voice activity detection (silence/pause detection)

Page 31 of 349

Text Feature Extraction

Library	Purpose
spaCy	Sentence segmentation, POS tagging, NER
NLTK	Tokenization, n-gram analysis
textstat	Readability scores, complexity metrics
scikit-learn	TF-IDF, cosine similarity (prompt relevance)

Infrastructure Stack (AWS)

Compute

- **Amazon ECS (Fargate)**: Container orchestration (serverless)
- **AWS Lambda**: Optional for lightweight processing tasks
- **Application Load Balancer (ALB)**: Traffic distribution

Storage

Page 32 of 349

- **Amazon S3**: Video/audio storage, static assets
- S3 Standard (frequent access)
- S3 Intelligent-Tiering (cost optimization)
- S3 Glacier (archival)
- **Amazon EFS**: Shared file system for ECS tasks (if needed)

Database

- **Amazon RDS (PostgreSQL)**: Managed relational database
- **Amazon ElastiCache (Redis)**: Managed caching

Networking

- **Amazon VPC**: Isolated network environment
- **CloudFront**: CDN for static assets and video streaming
- **Route 53**: DNS management

Page 33 of 349

Security

- **AWS WAF**: Web application firewall
- **AWS Secrets Manager**: Credential management
- **AWS KMS**: Encryption key management
- **AWS Certificate Manager**: SSL/TLS certificates

Monitoring & Logging

- **Amazon CloudWatch**: Metrics, logs, alarms
- **AWS X-Ray**: Distributed tracing
- **AWS CloudTrail**: API audit logging

CI/CD

Page 34 of 349

- **GitHub Actions**: Build and test automation
- **Amazon ECR**: Container registry
- **AWS CodeDeploy** or **GitHub Actions**: Deployment

DevOps & Tooling

Containerization

- **Docker**: Application containerization
- **Docker Compose**: Local development environment

Infrastructure as Code

- **Terraform**: AWS infrastructure provisioning
- **Terragrunt**: Terraform wrapper for DRY configs

Page 35 of 349

Version Control

- **Git**: Source control
- **GitHub**: Repository hosting, PR reviews
- **GitHub Actions**: CI/CD pipelines

Local Development

- **Docker Compose**: Full stack locally
- **pre-commit**: Git hooks for code quality
- **Makefile**: Common development tasks

Code Quality

- **Black**: Python code formatter

Page 36 of 349

- **isort**: Python import sorting
- **flake8**: Python linting
- **mypy**: Python type checking
- **pylint**: Additional Python linting

API Documentation

- **OpenAPI 3.0**: API specification
- **Swagger UI**: Interactive API docs
- **Redoc**: Alternative API docs

Package Managers

Frontend

Page 37 of 349

- **npm** or **pnpm**: JavaScript package manager (recommend pnpm)
- **package.json**: Dependency manifest

Backend

- **uv** or **poetry**: Modern Python package managers (recommend uv)
- **pyproject.toml**: Dependency and project metadata

Browser Support

Minimum Supported Versions

- Chrome 90+
- Firefox 88+

Page 38 of 349

- Safari 14+
- Edge 90+

Required Features

- MediaRecorder API
- WebRTC (for recording)
- ES2020+ JavaScript
- CSS Grid & Flexbox

Cost Considerations

Development (Est. Monthly)

Service	Cost
AWS ECS Fargate (dev)	$50-100

Page 39 of 349

RDS PostgreSQL (db.t3.micro)	$15
ElastiCache Redis	$15
S3 Storage	$5-20
OpenAI API	Usage-based
Total	**$100-200 + AI usage**

Production (Est. Monthly - Small Scale)

Service	Cost
AWS ECS Fargate	$200-500
RDS PostgreSQL (Multi-AZ)	$100-300
ElastiCache Redis	$50-100
S3 Storage + Transfer	$100-500
CloudFront CDN	$50-200
OpenAI API	$500-2000 (usage-based)
Total	**$1000-3500 + AI usage**

AWS Infrastructure

Overview

This document describes the AWS infrastructure architecture for Oracy AI, designed for scalability, security, and cost-efficiency.

VPC Architecture

Page 40 of 349

Network Layout

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                              VPC (10.0.0.0/16)                               │
│                                                                              │
│  ┌──────────────────────────────────────────────────────────────────────────┐
│  │                         PUBLIC SUBNETS                                   │
│  │  ┌────────────────────┐  ┌────────────────────┐  ┌────────────────────┐ │
│  │  │  Public Subnet A   │  │  Public Subnet B   │  │  Public Subnet C   │ │
│  │  │  (10.0.1.0/24)     │  │  (10.0.2.0/24)     │  │  (10.0.3.0/24)     │ │
│  │  │                    │  │                    │  │                    │ │
│  │  │  • ALB             │  │  • NAT Gateway B   │  │  • NAT Gateway C   │ │
│  │  │  • Bastion Host    │  │                    │  │                    │ │
│  │  └────────────────────┘  └────────────────────┘  └────────────────────┘ │
│  └──────────────────────────────────────────────────────────────────────────┘
│                                          │                                   │
│                                          │ IGW                               │
│                                          ▼                                   │
│  ┌──────────────────────────────────────────────────────────────────────────┐
│  │                         PRIVATE APP SUBNETS                              │
│  │  ┌────────────────────┐  ┌────────────────────┐  ┌────────────────────┐ │
│  │  │  App Subnet A      │  │  App Subnet B      │  │  App Subnet C      │ │
```


Page 41 of 349

```
│  │  │  (10.0.10.0/24)     │  │  (10.0.11.0/24)     │  │  (10.0.12.0/24)  │ │  │
│  │  │                     │  │                     │  │                  │ │  │
│  │  │  • ECS Fargate      │  │  • ECS Fargate      │  │  • ECS Fargate   │ │  │
│  │  │    (Django API)     │  │    (Celery Worker)  │  │    (Celery Beat) │ │  │
│  │  └─────────────────────┘  └─────────────────────┘  └──────────────────┘ │  │
│  └──────────────────────────────────────────────────────────────────────────┘  │
│                                          │                                      │
│  ┌──────────────────────────────────────────────────────────────────────────┐  │
│  │                        PRIVATE DATA SUBNETS                              │  │
│  │  ┌─────────────────────┐  ┌─────────────────────┐  ┌──────────────────┐ │  │
│  │  │  Data Subnet A      │  │  Data Subnet B      │  │  Data Subnet C   │ │  │
│  │  │  (10.0.20.0/24)     │  │  (10.0.21.0/24)     │  │  (10.0.22.0/24)  │ │  │
│  │  │                     │  │                     │  │                  │ │  │
│  │  │  • RDS PostgreSQL   │  │  • RDS PostgreSQL   │  │  • ElastiCache   │ │  │
│  │  │    (Primary)        │  │    (Standby)        │  │    Redis         │ │  │
│  │  └─────────────────────┘  └─────────────────────┘  └──────────────────┘ │  │
│  └──────────────────────────────────────────────────────────────────────────┘  │
│                                                                                 │
└─────────────────────────────────────────────────────────────────────────────────┘
```

Availability Zones

Page 42 of 349

- **3 AZs minimum** (ap-southeast-1a, ap-southeast-1b, ap-southeast-1c for Singapore)
- Resources distributed across AZs for high availability

Network Components

Component	CIDR	Purpose
VPC	10.0.0.0/16	Main network
Public Subnets	10.0.1.0/24, 10.0.2.0/24, 10.0.3.0/24	Load balancers, NAT gateways
Private App	10.0.10.0/24, 10.0.11.0/24, 10.0.12.0/24	ECS services
Private Data	10.0.20.0/24, 10.0.21.0/24, 10.0.22.0/24	Databases, cache

Security Groups

```
┌─────────────────────────────────────────────────────────────┐
│  ALB Security Group                                         │
│  Ingress:                                                   │
│    • 80/tcp from 0.0.0.0/0 (redirect to 443)                │
│    • 443/tcp from 0.0.0.0/0 (CloudFront or direct)          │
│  Egress: All                                                │
└─────────────────────────────────────────────────────────────┘
```


Page 43 of 349

```
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│  ECS Service Security Group                                  │
│  Ingress:                                                    │
│    • 8000/tcp from ALB SG                                    │
│  Egress:                                                     │
│    • 5432/tcp to RDS SG                                      │
│    • 6379/tcp to ElastiCache SG                              │
│    • 443/tcp to S3 API                                       │
│    • 443/tcp to external APIs (OpenAI, etc.)                 │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│  RDS Security Group                                          │
│  Ingress:                                                    │
│    • 5432/tcp from ECS Service SG                            │
│    • 5432/tcp from Bastion SG (admin)                        │
│  Egress: None (no outbound needed)                           │
└──────────────────────────────────────────────────────────────┘
```


Page 44 of 349

Compute Layer

ECS Fargate Cluster

Service Configuration

Service	Task CPU	Task Memory	Min	Max	Purpose
Django API	512	1024	2	10	Main API server
Celery Worker	1024	2048	2	20	AI pipeline processing
Celery Beat	256	512	1	1	Task scheduler
WebSocket	512	1024	2	5	Real-time notifications

Task Definitions

```
{
  "family": "oracy-django-api",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "512",
```


Page 45 of 349

```
"memory": "1024",
"executionRoleArn": "arn:aws:iam::ACCOUNT:role/ecsTaskExecutionRole",
"taskRoleArn": "arn:aws:iam::ACCOUNT:role/oracy-ecs-task-role",
"containerDefinitions": [
  {
    "name": "django",
    "image": "ACCOUNT.dkr.ecr.REGION.amazonaws.com/oracy-django:latest",
    "portMappings": [
      {
        "containerPort": 8000,
        "protocol": "tcp"
      }
    ],
    "environment": [
      {"name": "DJANGOSETTINGSMODULE", "value": "config.settings.production"},
      {"name": "DATABASEURL", "value": "${DATABASEURL}"},
      {"name": "REDISURL", "value": "${REDISURL}"}
    ],
    "secrets": [
      {
        "name": "SECRET_KEY",
        "valueFrom": "arn:aws:secretsmanager:REGION:ACCOUNT:secret:oracy/secret-key"
      },
```


Page 46 of 349

```
        {
          "name": "OPENAIAPIKEY",
          "valueFrom": "arn:aws:secretsmanager:REGION:ACCOUNT:secret:oracy/openai-key"
        }
      ],
      "logConfiguration": {
        "logDriver": "awslogs",
        "options": {
          "awslogs-group": "/ecs/oracy-django",
          "awslogs-region": "ap-southeast-1",
          "awslogs-stream-prefix": "ecs"
        }
      }
    }
  ]
}
```

Auto Scaling Policies

Django API Service:

- Target tracking on ALB request count per target (target: 1000)

Page 47 of 349

- Scale out: +2 tasks when CPU > 70% for 2 minutes
- Scale in: -1 task when CPU < 30% for 5 minutes
- Cooldown: 300 seconds

Celery Worker Service:

- Scale based on SQS queue depth (if using SQS) or Redis queue length
- Target queue depth: 100 messages per worker
- Max workers: 20 (to control AI API costs)

Database Layer

RDS PostgreSQL

Instance Configuration (MVP)

Page 48 of 349

Setting	Value
Engine	PostgreSQL 15.4
Instance Class	db.t3.medium (can scale to db.r6g.xlarge)
Multi-AZ	Yes (production) / No (dev)
Storage	100GB GP3 (auto-scaling enabled)
IOPS	3000
Backup Retention	7 days
Encryption	AWS KMS (AES-256)
Maintenance Window	Sunday 03:00-04:00 UTC

Parameter Group Settings

```
max_connections = 200

shared_buffers = {DBInstanceClassMemory/32768}

effectivecachesize = {DBInstanceClassMemory/8192}

workmem = {DBInstanceClassMemory/32768/maxconnections}

maintenanceworkmem = {DBInstanceClassMemory/8192}
```

Logging

```
log_statement = 'mod'

logminduration_statement = 1000

log_connections = on
```


Page 49 of 349

```
log_disconnections = on
```


Page 50 of 349

```
# Performance


randompagecost = 1.1

effectiveioconcurrency = 200
```

Read Replica (Future)

- Add read replica for reporting/analytics queries
- Cross-region replica for disaster recovery

ElastiCache Redis

Cluster Configuration

Setting	Value
Node Type	cache.t3.micro (dev) / cache.m6g.large (prod)
Engine	Redis 7.0
Cluster Mode	Disabled (single node for MVP, cluster later)
Multi-AZ	Yes
Auto-failover	Yes

Page 51 of 349

| Encryption | In-transit (TLS) and at-rest |
| Snapshot Retention | 1 day |

Redis Use Cases

1. **Session Storage**: Django session backend
2. **Celery Broker**: Task queue
3. **Celery Result Backend**: Task results
4. **API Cache**: Frequently accessed data (benchmarks, user profiles)
5. **Rate Limiting**: Django ratelimit backend

Storage Layer

S3 Buckets

Bucket Structure

```
oracy-{environment}-media              # Video/audio recordings
  ├── uploads/                         # Temporary upload staging
  ├── recordings/                      # Final processed recordings
  │   ├── {year}/                    # Organized by year
```


Page 52 of 349

```
  │   │   ├── {month}/
  │   │   │   ├── {assessment_id}/
  │   │   │   │   ├── video.mp4
  │   │   │   │   ├── audio.wav
  │   │   │   │   └── thumbnail.jpg
  ├── clips/                          # Evidence clips
  └── exports/                        # Generated PDFs


oracy-{environment}-assets            # Static assets (CSS, JS, images)
  ├── static/                         # Django collectstatic output
  └── media/                          # User-uploaded assets




oracy-{environment}-logs              # Application logs
  ├── app-logs/                       # Django application logs
  ├── alb-logs/                       # Load balancer access logs
  └── cloudtrail/                     # AWS API audit logs




oracy-{environment}-backups           # Database backups
  ├── rds-snapshots/                  # Automated RDS backups
```


Page 53 of 349

```
  └── manual/                          # Manual backups
```

S3 Security Configuration

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyNonSSL",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:*",
      "Resource": [
        "arn:aws:s3:::oracy-prod-media/*",
        "arn:aws:s3:::oracy-prod-media"
      ],
      "Condition": {
        "Bool": {
          "aws:SecureTransport": "false"
        }
      }
```


Page 54 of 349

```
    },
    {
      "Sid": "AllowECSTaskRole",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::ACCOUNT:role/oracy-ecs-task-role"
      },
      "Action": [
        "s3:PutObject",
        "s3:GetObject",
        "s3:DeleteObject"
      ],
      "Resource": "arn:aws:s3:::oracy-prod-media/*"
    }
  ]
}
```

S3 Lifecycle Policies

Bucket Path	Transition to IA	Transition to Glacier	Expiration
uploads/*	1 day	-	7 days (temporary)

Page 55 of 349

recordings/*	30 days	90 days	Never (7yr retention)
clips/*	30 days	90 days	Never
exports/*	7 days	30 days	365 days

CloudFront Distribution

Distribution Configuration

Setting	Value
Origins	S3 (assets), ALB (API), S3 (media streaming)
Price Class	PriceClass*All (global) or PriceClass*100 (Asia focus)
SSL	ACM certificate (oracy.ai, *.oracy.ai)
Compression	Brotli and Gzip
HTTP/2 & HTTP/3	Enabled
WAF	AWS WAF Web ACL attached

Cache Behaviors

Path Pattern	Origin	TTL	Forward Headers
/api/*	ALB	0 (no cache)	All
/ws/*	ALB	0 (WebSocket)	All
/static/*	S3 Assets	1 year	None
/media/recordings/*	S3 Media	1 hour (signed URLs)	Authorization
/*	S3 Assets	Default	None

Page 56 of 349

Signed URLs for Video

All video access through CloudFront with signed URLs:

- URL expiration: 1 hour
- Restrict viewer access: Yes
- Trusted signer: Or account

AI/ML Services (AWS Native)

AWS Transcribe (Optional/Fallback)

Usage Pattern

```
# For high-accuracy fallback or batch processing

import boto3
```


Page 57 of 349

```
transcribe = boto3.client('transcribe')




response = transcribe.starttranscriptionjob(
    TranscriptionJobName=f"oracy-{assessment_id}",
    Media={'MediaFileUri': f's3://oracy-media/recordings/{path}'},
    MediaFormat='mp4',
    LanguageCode='en-GB',  # or en-US
    Settings={
        'ShowSpeakerLabels': False,
        'MaxSpeakerLabels': 1,
        'ChannelIdentification': False,
        'ShowAlternatives': False,
    },
    OutputBucketName='oracy-transcripts',
    OutputKey=f'{assessment_id}/transcript.json'
)
```

Amazon SageMaker (Future Custom Models)

Page 58 of 349

Reserved for Phase 2 custom model training:

- Notebook instances for experimentation
- Training jobs for rubric classifier
- Endpoints for model serving
- Model monitoring and drift detection

Security & IAM

IAM Roles

ECS Task Execution Role

```
{
  "Version": "2012-10-17",
```


Page 59 of 349

```
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ecr:GetAuthorizationToken",
        "ecr:BatchCheckLayerAvailability",
        "ecr:GetDownloadUrlForLayer",
        "ecr:BatchGetImage",
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ],
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "secretsmanager:GetSecretValue"
      ],
      "Resource": "arn:aws:secretsmanager:::secret:oracy/*"
    }
  ]
}
```


Page 60 of 349

ECS Task Role

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:PutObject",
        "s3:GetObject",
        "s3:DeleteObject"
      ],
      "Resource": [
        "arn:aws:s3:::oracy--media/",
        "arn:aws:s3:::oracy--exports/"
      ]
    },
    {
      "Effect": "Allow",
      "Action": [
        "transcribe:StartTranscriptionJob",
```


Page 61 of 349

```
        "transcribe:GetTranscriptionJob"
      ],
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "cloudwatch:PutMetricData",
        "xray:PutTraceSegments",
        "xray:PutTelemetryRecords"
      ],
      "Resource": "*"
    }
  ]
}
```

Secrets Manager

Secrets stored in AWS Secrets Manager:

- `oracy/secret-key` - Django SECRET_KEY

Page 62 of 349

- `oracy/openai-key` - OpenAI API key
- `oracy/anthropic-key` - Anthropic API key
- `oracy/database-password` - RDS master password
- `oracy/jwt-signing-key` - JWT signing key

KMS Keys

Key Alias	Purpose
`alias/oracy-rds`	RDS encryption
`alias/oracy-s3`	S3 bucket encryption
`alias/oracy-secrets`	Secrets Manager encryption
`alias/oracy-ebs`	EBS volume encryption

Monitoring & Logging

CloudWatch

Page 63 of 349

Log Groups

- `/ecs/oracy-django` - Django application logs
- `/ecs/oracy-celery` - Celery worker logs
- `/aws/alb/oracy` - Load balancer access logs
- `/aws/rds/oracy` - RDS slow query logs

Metrics & Alarms

Metric	Threshold	Action
ECS CPU Utilization	> 80% for 5 min	Scale out
ECS Memory Utilization	> 85% for 5 min	Alert + Review
RDS CPU Utilization	> 80% for 10 min	Scale up instance
RDS FreeableMemory	< 100MB	Alert immediately
S3 5xx Errors	> 10 in 5 min	Alert + Investigate
ALB 5xx Errors	> 1% of requests	Alert + Investigate

X-Ray

Enable distributed tracing:

- Django middleware for request tracing

Page 64 of 349

- Celery task tracing
- AWS SDK instrumentation (boto3)

CloudTrail

- Log all API calls to S3, RDS, IAM
- Store logs in S3 with Glacier transition
- Enable log file validation

Cost Optimization

Reserved Capacity

- RDS Reserved Instances (1-year, no upfront)

Page 65 of 349

- Savings Plans for Fargate compute

Spot Instances (Optional)

- Use Fargate Spot for Celery workers (fault-tolerant)
- Up to 70% cost savings

S3 Intelligent Tiering

- Automatically move infrequently accessed videos to cheaper tiers

CloudFront Caching

- Aggressive caching for static assets
- Reduce origin requests by 90%+

AI/ML Pipeline

Page 66 of 349

Overview

The Oracy AI assessment pipeline consists of 5 sequential stages, transforming raw video/audio into structured, teacher-verifiable assessment reports. Each stage is modular and can be independently versioned, scaled, or replaced.

```
┌──────────────
```

```
│   │  • 3 strengths + 3 next steps (mode-specific)                          │   │
│   │  • 1-2 student-friendly practice goals                                 │   │
│   │  • Confidence indicators per strand                                    │   │
│   └────────────────────────────────────────────────────────────────────────┘   │
│
└──────────────────────────────────────────────────────────────────────────────────
```

Step 1: Speech-to-Text (STT)

Purpose

Generate a timestamped transcript from the audio track with word-level or segment-level timestamps.

Input

- Audio file (WAV or MP4 audio track)
- Recording ID for correlation

Output


```
{
  "assessment_id": "uuid",
  "language": "en-GB",
  "duration_seconds": 124.5,
  "segments": [
    {
      "id": 0,
      "start": 0.0,
      "end": 4.2,
      "text": "Today I'm going to explain photosynthesis.",
      "words": [
        {"word": "Today", "start": 0.0, "end": 0.4, "confidence": 0.98},
        {"word": "I'm", "start": 0.5, "end": 0.7, "confidence": 0.95},
        {"word": "going", "start": 0.8, "end": 1.0, "confidence": 0.97}
      ]
    }
  ],
  "full_text": "Today I'm going to explain photosynthesis...",
```


Page 69 of 349

```
  "confidence_overall": 0.94

}
```

Implementation Options

Option A: OpenAI Whisper API (Recommended for MVP)

```
import openai



class WhisperSTTService:

    def init(self, api_key: str):

        self.client = openai.OpenAI(apikey=apikey)



    def transcribe(self, audio_path: str) -> Transcript:

        with open(audiopath, 'rb') as audiofile:

            response = self.client.audio.transcriptions.create(

                model="gpt-4o-transcribe",  # or "whisper-1"

                file=audio_file,

                responseformat="verbosejson",

                timestamp_granularities=["word", "segment"]
```


Page 70 of 349

```
            )


        return self.parseresponse(response)


    def parseresponse(self, response) -> Transcript:

        # Convert OpenAI format to internal format

        return Transcript(

            segments=[

                Segment(

                    start=seg.start,

                    end=seg.end,

                    text=seg.text,

                    words=[

                        Word(start=w.start, end=w.end, word=w.word, confidence=w.proba

                        for w in seg.words

                    ]

                )

                for seg in response.segments

            ],

            confidenceoverall=response.avglogprob

        )
```


Page 71 of 349

Pros:

- High accuracy across accents and ages
- Built-in timestamp support
- Cost-effective
- No infrastructure to manage

Cons:

- External dependency
- Data leaves AWS (consider for GDPR)

Option B: AWS Transcribe (Alternative)

```
import boto3



class AWSTranscribeService:

    def init(self):

        self.client = boto3.client('transcribe')
```


Page 72 of 349

```
def transcribe(self, audios3uri: str, job_name: str) -> str:
    # Start async transcription job
    self.client.starttranscriptionjob(
        TranscriptionJobName=job_name,
        Media={'MediaFileUri': audios3uri},
        MediaFormat='mp4',
        LanguageCode='en-GB',
        OutputBucketName='oracy-transcripts',
        Settings={
            'ShowSpeakerLabels': False,
            'ChannelIdentification': False
        }
    )

    # Poll for completion (in Celery task)
    return job_name
```

Pros:

- Native AWS integration

Page 73 of 349

- HIPAA eligible
- Speaker diarization (future)

Cons:

- Async only (adds latency)
- Higher cost for short files
- Less accurate for child voices

Option C: Self-Hosted Whisper (Future)

Use `faster-whisper` or `whisper.cpp` on ECS GPU instances for:

- Full data sovereignty
- Predictable costs at scale
- Custom fine-tuning

Service Interface

Page 74 of 349

Abstract STT behind an interface for easy swapping:

```
from abc import ABC, abstractmethod

from typing import Protocol



class STTService(Protocol):

    @abstractmethod

    def transcribe(self, audio_path: str) -> Transcript:

        """Generate timestamped transcript."""

        ...



    @abstractmethod

    def supports_streaming(self) -> bool:

        """Whether real-time transcription is supported."""

        ...
```

Usage

```
sttservice: STTService = WhisperSTTService(apikey)  # or AWSTranscribeService()
```


Page 75 of 349

```
transcript = sttservice.transcribe(audiopath)
```

Step 2: Deterministic Feature Extraction

Purpose

Extract measurable, objective signals from audio and transcript WITHOUT using LLMs. These features serve as inputs to the benchmarking layer.

Philosophy

- **Do NOT use LLMs for raw signal measurement**
- Use established audio processing and NLP libraries
- Store all extracted features for auditability and future model training

Physical Features (Audio)

Page 76 of 349

Feature	Method	Library
Words per minute (WPM)	Word count / duration	librosa + transcript
Pause ratio	Silence detection / total duration	webrtcvad
Filler word frequency	Regex matching on transcript	re (built-in)
Volume variability	RMS energy variance	librosa
Rhythm stability	Tempo estimation variance	librosa.beat
Pitch variation	F0 (fundamental frequency) std	librosa

```
import librosa

import numpy as np

from pydub import AudioSegment

import webrtcvad



class PhysicalFeatureExtractor:

    def init(self):

        self.vad = webrtcvad.Vad(2)  # Aggressiveness 0-3

        self.filler_words = {'um', 'uh', 'er', 'ah', 'like', 'you know'}



    def extract(self, audio_path: str, transcript: Transcript) -> PhysicalFeatures:

        # Load audio

        y, sr = librosa.load(audio_path, sr=None)

        duration = librosa.get_duration(y=y, sr=sr)
```


Page 77 of 349

```
        # Words per minute
        wordcount = len(transcript.allwords())
        wpm = (word_count / duration) * 60


        # Pause analysis using VAD
        pauses = self.detectpauses(y, sr)
        pause_ratio = sum(p['duration'] for p in pauses) / duration


        # Filler words
        filler_count = sum(
            1 for word in transcript.all_words()
            if word.lower() in self.filler_words
        )
        fillerfrequency = fillercount / duration  # per second


        # Volume variability (RMS energy)
        rms = librosa.feature.rms(y=y)[0]
        volume_variance = np.var(rms)


        # Rhythm/tempo stability
        tempo, beatframes = librosa.beat.beattrack(y=y, sr=sr)
        beatintervals = np.diff(librosa.framestotime(beatframes, sr=sr))
        rhythmstability = 1.0 / (np.std(beatintervals) + 1e-6)
```


Page 78 of 349

```
        return PhysicalFeatures(
            wpm=wpm,
            pauseratio=pauseratio,
            fillerfrequency=fillerfrequency,
            volumevariance=volumevariance,
            rhythmstability=rhythmstability
        )

    def detectpauses(self, y: np.ndarray, sr: int) -> List[Pause]:
        # Convert to pcm16 for webrtcvad
        pcm_data = (y * 32767).astype(np.int16).tobytes()

        # Frame duration must be 10, 20, or 30ms
        frameduration ms = 30
        framelength = int(sr * frameduration_ms / 1000)

        pauses = []
        is_speech = False
        pause_start = 0

        for i in range(0, len(pcmdata), framelength * 2):  # 2 bytes per sample
            frame = pcmdata[i:i + framelength * 2]
```


Page 79 of 349

```
            if len(frame) < frame_length * 2:
                break

            try:
                speech = self.vad.is_speech(frame, sr)
            except:
                speech = False

            if not speech and is_speech:
                pause_start = i / (sr * 2)  # Convert to seconds
                is_speech = False
            elif speech and not is_speech:
                pause_end = i / (sr * 2)
                if pause_end - pause_start > 0.5:  # Minimum 500ms pause
                    pauses.append(Pause(start=pause_start, end=pause_end))
                is_speech = True

        return pauses
```

Linguistic Features (Transcript)

Page 80 of 349

Feature	Method	Library
Sentence length variance	Parse sentences, calculate std	spaCy
Vocabulary diversity	Type-Token Ratio (TTR)	spaCy
Prompt relevance	TF-IDF cosine similarity	scikit-learn
Clarity markers	Flesch Reading Ease score	textstat
Register formality	Formality index	Custom rules

```
import spacy

from textstat import flesch_reading_ease

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.metrics.pairwise import cosine_similarity



class LinguisticFeatureExtractor:

    def init(self):

        self.nlp = spacy.load('en_core_web_sm')

        self.vectorizer = TfidfVectorizer(stop_words='english')



    def extract(self, transcript: Transcript, prompt: str) -> LinguisticFeatures:

        full_text = transcript.full_text

        doc = self.nlp(full_text)



        # Sentence length variance
```


Page 81 of 349

```
        sentences = list(doc.sents)

        sent_lengths = [len(sent) for sent in sentences]

        sentencelengthvariance = np.var(sentlengths) if len(sentlengths) > 1 else 0



        # Vocabulary diversity (TTR)

        tokens = [token.text.lower() for token in doc if not token.is_punct]

        unique_tokens = set(tokens)

        ttr = len(unique_tokens) / len(tokens) if tokens else 0



        # Prompt relevance

        promptrelevance = self.calculaterelevance(fulltext, prompt)



        # Clarity (Flesch Reading Ease)

        # Higher = easier to read

        clarityscore = fleschreadingease(fulltext)



        # Register detection (simple heuristic)

        formal_markers = ['furthermore', 'however', 'therefore', 'consequently']

        informal_markers = ['like', 'yeah', 'gonna', 'wanna']



        formalcount = sum(1 for m in formalmarkers if m in full_text.lower())

        informalcount = sum(1 for m in informalmarkers if m in full_text.lower())

        registerscore = (formalcount - informal_count) / len(sentences) if sentences e
```


Page 82 of 349

```
        return LinguisticFeatures(
            sentencelengthvariance=sentencelengthvariance,
            vocabulary_diversity=ttr,
            promptrelevance=promptrelevance,
            clarityscore=clarityscore,
            registerformality=registerscore
        )


    def calculaterelevance(self, text: str, prompt: str) -> float:
        # Simple TF-IDF cosine similarity
        try:
            tfidf = self.vectorizer.fit_transform([text, prompt])
            return float(cosine_similarity(tfidf[0:1], tfidf[1:2])[0][0])
        except:
            return 0.5  # Neutral if calculation fails
```

Cognitive Features (Transcript)

Feature	Method	Markers
Reason markers	Keyword frequency	"because", "for example", "such as"

Page 83 of 349

Structure signals	Section detection	Intro/body/conclusion markers
Counterpoint markers	Keyword frequency	"however", "although", "on the other hand"
Logical connectors	Keyword frequency	"firstly", "then", "finally", "therefore"
Coherence	Entity chain analysis	spaCy coreference

```
class CognitiveFeatureExtractor:

    REASON_MARKERS = ['because', 'since', 'as', 'for example', 'such as', 'like']

    COUNTERPOINT_MARKERS = ['however', 'although', 'though', 'but', 'yet', 'on the oth

    LOGICAL_CONNECTORS = ['first', 'firstly', 'second', 'secondly', 'then', 'next', 'f

    STRUCTURE_MARKERS = {

        'intro': ['today', "i'm going to", "i will", "let me explain", "i'd like to"],

        'conclusion': ['in conclusion', 'to summarize', 'in summary', 'finally', 'over

    }



    def extract(self, transcript: Transcript) -> CognitiveFeatures:

        textlower = transcript.fulltext.lower()

        wordcount = len(transcript.allwords())



        # Reason density

        reasoncount = sum(textlower.count(m) for m in self.REASON_MARKERS)

        reasondensity = reasoncount / wordcount if wordcount > 0 else 0



        # Counterpoint usage

        counterpointcount = sum(textlower.count(m) for m in self.COUNTERPOINT_MARKERS)
```


Page 84 of 349

```
        counterpointdensity = counterpointcount / wordcount if wordcount > 0 else 0



        # Logical connector usage

        connectorcount = sum(textlower.count(m) for m in self.LOGICAL_CONNECTORS)

        connectordensity = connectorcount / wordcount if wordcount > 0 else 0



        # Structure detection

        hasintro = any(m in textlower for m in self.STRUCTURE_MARKERS['intro'])

        hasconclusion = any(m in textlower for m in self.STRUCTURE_MARKERS['conclusion

        structurescore = (int(hasintro) + int(has_conclusion)) / 2



        return CognitiveFeatures(

            reasondensity=reasondensity,

            counterpointdensity=counterpointdensity,

            logicalconnectordensity=connector_density,

            structurecompleteness=structurescore

        )
```

Social-Emotional Features (Solo Mode)

Page 85 of 349

Feature	Method	Markers
Audience reference	Keyword frequency	"you", "your", "we", "our"
Intention clarity	Confidence markers	"I believe", "I think", "definitely"
Tone markers	Sentiment + intensity	spaCy sentiment or VADER

```
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer



class SocialEmotionalFeatureExtractor:

    def init(self):

        self.sentiment_analyzer = SentimentIntensityAnalyzer()



    def extract(self, transcript: Transcript) -> SocialEmotionalFeatures:

        text = transcript.full_text.lower()

        wordcount = len(transcript.allwords())



        # Audience reference frequency

        audience_refs = ['you', 'your', 'we', 'our', 'us']

        audiencecount = sum(text.count(f' {ref} ') + text.count(f' {ref}.') for ref in

        audiencereferencefrequency = audiencecount / wordcount if word_count > 0 else



        # Intention clarity (confidence markers)

        confidence_markers = ['definitely', 'certainly', 'absolutely', 'clearly', 'obv

        hedging_markers = ['maybe', 'perhaps', 'possibly', 'might', 'could be']
```


Page 86 of 349

```
        confidencecount = sum(text.count(m) for m in confidencemarkers)

        hedgingcount = sum(text.count(m) for m in hedgingmarkers)

        intentionclarity = (confidencecount - hedgingcount) / wordcount if word_count


        # Tone analysis

        sentiment = self.sentimentanalyzer.polarityscores(transcript.full_text)


        return SocialEmotionalFeatures(

            audiencereferencefrequency=audiencereferencefrequency,

            intentionclarity=intentionclarity,

            sentiment_positive=sentiment['pos'],

            sentiment_negative=sentiment['neg'],

            sentiment_neutral=sentiment['neu'],

            sentiment_compound=sentiment['compound']

        )
```

Feature Storage

All extracted features stored in database:

Page 87 of 349

```
class FeatureSignals(models.Model):

    assessment = models.OneToOneField(Assessment, on_delete=models.CASCADE)



    # Physical

    wpm = models.FloatField()

    pause_ratio = models.FloatField()

    filler_frequency = models.FloatField()

    volume_variance = models.FloatField()

    rhythm_stability = models.FloatField()



    # Linguistic

    sentencelengthvariance = models.FloatField()

    vocabulary_diversity = models.FloatField()

    prompt_relevance = models.FloatField()

    clarity_score = models.FloatField()

    register_formality = models.FloatField()



    # Cognitive

    reason_density = models.FloatField()

    counterpoint_density = models.FloatField()

    logicalconnectordensity = models.FloatField()
```


Page 88 of 349

```
    structure_completeness = models.FloatField()



    # Social-emotional

    audiencereferencefrequency = models.FloatField()

    intention_clarity = models.FloatField()

    sentiment_compound = models.FloatField()



    # Metadata

    extractedat = models.DateTimeField(autonow_add=True)

    qualityflag = models.CharField(maxlength=20)  # 'good', 'fair', 'poor'
```

Step 3: Benchmarking Layer (Core IP)

Purpose

The Benchmarking Layer is the core intellectual property of Oracy AI. It translates extracted features into band levels (Emerging/Expected/Exceeding) based on age-appropriate, mode-specific rubrics.

Benchmark Definition Language (BDL)

Page 89 of 349

JSON-based structured format for rubric definitions:

```
{
  "version": "1.0.0",
  "age_band": "11-12",
  "mode": "explaining",
  "created_at": "2026-03-01",
  "strands": {
    "physical": {
      "subskills": [
        {
          "name": "pace and fluency",
          "weight": 0.4,
          "bands": {
            "emerging": {
              "description": "Frequent pauses, hesitations, or rushed speech that hind
              "evidence_conditions": [
                {"feature": "wpm", "operator": "not_between", "values": [100, 160]},
                {"feature": "pause_ratio", "operator": ">", "value": 0.3}
              ],
              "disallowed_shortcuts": ["reading from notes without adaptation"]
```


Page 90 of 349

```
        },
        "expected": {
          "description": "Generally steady pace with appropriate pauses for unders
          "evidence_conditions": [
            {"feature": "wpm", "operator": "between", "values": [110, 150]},
            {"feature": "pause_ratio", "operator": "between", "values": [0.15, 0.2
          ]
        },
        "exceeding": {
          "description": "Varied pace used deliberately for emphasis and engagemen
          "evidence_conditions": [
            {"feature": "wpm", "operator": "between", "values": [120, 160]},
            {"feature": "pause_ratio", "operator": "between", "values": [0.2, 0.3]
            {"feature": "rhythm_stability", "operator": ">", "value": 0.7}
          ]
        }
      }
    },
    {
      "name": "vocal_clarity",
      "weight": 0.3,
      "bands": { ... }
    },
```


Page 91 of 349

```
        {
          "name": "useoffillers",
          "weight": 0.3,
          "bands": { ... }
        }
      ]
    },
    "linguistic": { ... },
    "cognitive": { ... },
    "social_emotional": { ... }
  }
}
```

BDL Interpreter

```
from typing import Dict, List, Any
import operator


class BenchmarkEngine:
    """Interprets BDL and applies to extracted features."""
```


Page 92 of 349

```
OPERATORS = {
    '>': operator.gt,
    '<': operator.lt,
    '>=': operator.ge,
    '<=': operator.le,
    '==': operator.eq,
    'between': lambda x, vals: vals[0] <= x <= vals[1],
    'not_between': lambda x, vals: not (vals[0] <= x <= vals[1])
}


def init(self, bdl_path: str):
    self.bdl = self.loadbdl(bdl_path)


def evaluate(self, strand: str, features: Dict[str, float]) -> StrandEvaluation:
    """Evaluate a single strand against benchmarks."""
    strand_def = self.bdl['strands'][strand]


    subskill_scores = []
    for subskill in strand_def['subskills']:
        bandscores = self.score_subskill(subskill, features)
        bestband = max(bandscores, key=band_scores.get)
        subskill_scores.append({
```


Page 93 of 349

```
        'name': subskill['name'],
        'band': best_band,
        'confidence': bandscores[bestband],
        'weight': subskill['weight']
    })

# Weighted aggregation
band_weights = {'emerging': 1, 'expected': 2, 'exceeding': 3}
weighted_score = sum(
    band_weights[s['band']]  s['weight']  s['confidence']
    for s in subskill_scores
) / sum(s['weight'] for s in subskill_scores)

# Map back to band
if weighted_score < 1.5:
    overall_band = 'emerging'
elif weighted_score < 2.5:
    overall_band = 'expected'
else:
    overall_band = 'exceeding'

return StrandEvaluation(
    strand=strand,
```


Page 94 of 349

```
            band=overall_band,
            confidence=self.calculateconfidence(subskill_scores),
            subskills=subskill_scores
        )


    def scoresubskill(self, subskill: Dict, features: Dict) -> Dict[str, float]:
        """Score a subskill against all bands."""
        scores = {}
        for bandname, banddef in subskill['bands'].items():
            score = self.evaluateconditions(banddef['evidenceconditions'], features)
            scores[band_name] = score
        return scores


    def evaluateconditions(self, conditions: List[Dict], features: Dict) -> float:
        """Evaluate evidence conditions and return match score (0-1)."""
        if not conditions:
            return 0.5


        matches = 0
        for condition in conditions:
            feature_value = features.get(condition['feature'])
            if feature_value is None:
                continue
```


Page 95 of 349

```
            op = self.OPERATORS[condition['operator']]


            if 'values' in condition:
                if op(feature_value, condition['values']):
                    matches += 1
            elif op(feature_value, condition['value']):
                matches += 1


        return matches / len(conditions) if conditions else 0.5
```

Versioning

Benchmarks are versioned and immutable:

```
class BenchmarkVersion(models.Model):
    version = models.CharField(max_length=20, primary_key=True)
    age_band = models.CharField(max_length=10)
    mode = models.CharField(max_length=20)
    definition = models.JSONField()  # Full BDL
```


Page 96 of 349

```
    is_active = models.BooleanField(default=True)

    createdat = models.DateTimeField(autonow_add=True)



    class Meta:

        uniquetogether = ['ageband', 'mode', 'version']
```

Step 4: Evidence Candidate Generation

Purpose

Generate 10-30 candidate video segments that COULD serve as evidence for the assessment. The LLM is constrained to choose from these candidates only.

Candidate Generation Strategy

```
class EvidenceCandidateGenerator:

    """Generate timestamped evidence candidates from transcript and features."""
```


Page 97 of 349

```
def generate(
    self,
    transcript: Transcript,
    features: FeatureSignals,
    target_count: int = 20
) -> List[EvidenceCandidate]:
    """Generate diverse evidence candidates."""

    candidates = []

    # 1. Transcript boundaries (natural breaks)
    boundary_candidates = self.from_transcript_boundaries(transcript)
    candidates.extend(boundary_candidates)

    # 2. High reason-density segments
    reason_candidates = self.from_reason_density(transcript)
    candidates.extend(reason_candidates)

    # 3. Filler-heavy sections (areas for improvement)
    filler_candidates = self.from_filler_sections(transcript)
    candidates.extend(filler_candidates)

    # 4. Volume variance sections (emphasis or uncertainty)
```


Page 98 of 349

```
        volumecandidates = self.fromvolumesections(transcript, features)
        candidates.extend(volume_candidates)

        # 5. Prompt relevance peaks
        relevancecandidates = self.fromprompt relevance(transcript)
        candidates.extend(relevance_candidates)

        # Remove overlaps and limit
        candidates = self.deduplicateandselect(candidates, targetcount)

        return candidates

    def fromtranscript_boundaries(self, transcript: Transcript) -> List[EvidenceCandid
        """Generate candidates from natural speech boundaries."""
        candidates = []

        for segment in transcript.segments:
            # Only segments of reasonable length (10-30 seconds)
            duration = segment.end - segment.start
            if 10 <= duration <= 30:
                candidates.append(EvidenceCandidate(
                    start=segment.start,
                    end=segment.end,
```


Page 99 of 349

```
                type='transcript_boundary',
                summary=segment.text[:200],
                features={'duration': duration, 'word_count': len(segment.words)}
            ))

    return candidates

def fromreason_density(self, transcript: Transcript) -> List[EvidenceCandidate]:
    """Find segments with high reasoning density."""
    reason_markers = ['because', 'for example', 'such as', 'therefore']
    candidates = []

    # Sliding window over transcript
    window_size = 20  # words
    stride = 10

    words = transcript.all_words()
    for i in range(0, len(words) - window_size, stride):
        window = words[i:i + window_size]
        window_text = ' '.join([w.word for w in window]).lower()

        reasoncount = sum(windowtext.count(m) for m in reason_markers)
        density = reasoncount / windowsize
```


Page 100 of 349

```
        if density > 0.15:  # Threshold for high reasoning
            start = window[0].start
            end = window[-1].end

            # Expand to make 10-30 second clips
            if end - start < 10:
                end = min(start + 10, transcript.duration)
            elif end - start > 30:
                end = start + 30

            candidates.append(EvidenceCandidate(
                start=start,
                end=end,
                type='reason_dense',
                summary=window_text[:200],
                features={'reason_density': density}
            ))

    return candidates

def deduplicateand_select(
    self,
```


Page 101 of 349

```
        candidates: List[EvidenceCandidate],
        target_count: int
    ) -> List[EvidenceCandidate]:
        """Remove overlapping candidates and select diverse set."""
        # Sort by type to ensure diversity
        by_type = defaultdict(list)
        for c in candidates:
            by_type[c.type].append(c)

        selected = []
        typecycle = cycle(bytype.keys())

        while len(selected) < targetcount and any(bytype.values()):
            typekey = next(typecycle)
            if bytype[typekey]:
                candidate = bytype[typekey].pop(0)

                # Check for overlap with selected
                overlaps = any(
                    self.segmentsoverlap(candidate, s)
                    for s in selected
                )
```


Page 102 of 349

```
            if not overlaps:
                selected.append(candidate)

    return selected

def segmentsoverlap(self, a: EvidenceCandidate, b: EvidenceCandidate) -> bool:
    """Check if two segments overlap significantly (>50%)."""
    overlap_start = max(a.start, b.start)
    overlap_end = min(a.end, b.end)

    if overlapstart >= overlapend:
        return False

    overlapduration = overlapend - overlap_start
    a_duration = a.end - a.start
    b_duration = b.end - b.start

    return overlapduration > min(aduration, b_duration) * 0.5
```


Page 103 of 349

Step 5: Rubric-Constrained LLM Scoring

Purpose

Use an LLM to:
1. Map signals + transcript to band per strand
2. Select valid evidence clips (from candidates only)
3. Generate justification text
4. Produce strengths, next steps, and goals

Constraints

- **MUST NOT invent timestamps** - only use provided candidates
- **MUST use structured JSON output**
- **MUST reference rubric descriptors**
- **MUST output confidence scores**
- **MUST state if insufficient evidence**

Prompt Engineering

Page 104 of 349

```
RUBRICSCORINGPROMPT = """You are an expert oracy assessor evaluating a student's spoke
```

ASSESSMENT CONTEXT

- Student Age Band: {age_band}
- Mode of Talk: {mode}
- Prompt/Topic: {prompt}

BENCHMARK DEFINITIONS

{benchmark_json}

EXTRACTED FEATURES

{features_json}

Page 105 of 349

TRANSCRIPT

```
{transcript_text}
```

EVIDENCE CANDIDATES (MUST select from these ONLY)

```
{candidates_json}
```

SCORING INSTRUCTIONS

```
For each of the 4 strands (physical, linguistic, cognitive, social-emotional):

1. Evaluate against the benchmark definitions for this age band and mode
2. Assign a band level: "emerging", "expected", or "exceeding"
3. Select 3-8 evidence clip IDs from the candidates above
4. Write a justification (2-3 sentences) referencing specific rubric language
5. Assign a confidence score (0.0-1.0) based on evidence quality
```


Page 106 of 349

If there is insufficient evidence for a strand, set band to "insufficient_evidence" an

MODE-SPECIFIC FEEDBACK

Generate feedback that is SPECIFIC to the {mode} mode:

- 3 strengths with mode-specific language (e.g., for "persuading": "effective use of
- 3 next steps tied to mode requirements
- 1-2 student-friendly practice goals

OUTPUT FORMAT

Page 107 of 349

```
Respond with ONLY a valid JSON object matching this schema:




{{
  "physical": {{
    "band": "emerging|expected|exceeding|insufficient_evidence",
    "confidence": 0.0-1.0,
    "evidenceclipids": ["id1", "id2", ...],
    "justification": "string referencing rubric descriptors"
  }},
  "linguistic": {{ ... }},
  "cognitive": {{ ... }},
  "social_emotional": {{ ... }},
  "feedback": {{
    "strengths": ["string1", "string2", "string3"],
    "next_steps": ["string1", "string2", "string3"],
    "goals": ["string1"]
  }}
}}
```


Page 108 of 349

```
"""
```

Implementation

```
import json

from openai import OpenAI



class LLMScoringService:

    def init(self, api_key: str):

        self.client = OpenAI(apikey=apikey)



    def score(

        self,

        assessment: Assessment,

        features: FeatureSignals,

        transcript: Transcript,

        candidates: List[EvidenceCandidate],

        benchmark: BenchmarkVersion

    ) -> DraftReport:

        """Generate AI draft report using LLM."""
```


Page 109 of 349

```
        # Prepare candidates JSON with IDs
        candidates_json = [
            {
                'id': f'clip_{i}',
                'start': c.start,
                'end': c.end,
                'type': c.type,
                'summary': c.summary
            }
            for i, c in enumerate(candidates)
        ]

        prompt = RUBRICSCORINGPROMPT.format(
            ageband=assessment.ageband,
            mode=assessment.mode,
            prompt=assessment.prompt,
            benchmark_json=json.dumps(benchmark.definition, indent=2),
            featuresjson=self.featurestojson(features),
            transcripttext=transcript.fulltext[:5000],  # Limit context
            candidatesjson=json.dumps(candidatesjson, indent=2)
        )
```


Page 110 of 349

```
        response = self.client.chat.completions.create(
            model="gpt-4o",  # or "gpt-4o-2024-08-06"
            messages=[
                {
                    "role": "system",
                    "content": "You are a precise oracy assessment evaluator. Always r
                },
                {"role": "user", "content": prompt}
            ],
            responseformat={"type": "jsonobject"},
            temperature=0.3,  # Low temperature for consistency
            max_tokens=4000
        )

        result = json.loads(response.choices[0].message.content)

        # Validate and map clip IDs back to timestamps
        validated = self.validateandmapclips(result, candidates)

        return DraftReport(
            assessment=assessment,
            physical_score=validated['physical'],
            linguistic_score=validated['linguistic'],
```


Page 111 of 349

```
            cognitive_score=validated['cognitive'],
            socialemotionalscore=validated['social_emotional'],
            feedback=validated['feedback'],
            generated_at=timezone.now()
        )


    def validateandmapclips(
        self,
        result: Dict,
        candidates: List[EvidenceCandidate]
    ) -> Dict:
        """Validate clip IDs exist and map to actual timestamps."""
        candidatesbyid = {f'clip_{i}': c for i, c in enumerate(candidates)}


        for strand in ['physical', 'linguistic', 'cognitive', 'social_emotional']:
            strand_data = result[strand]
            validclipids = [
                cid for cid in stranddata['evidenceclip_ids']
                if cid in candidatesbyid
            ]


            # If LLM hallucinated clip IDs, flag with low confidence
            if len(validclipids) < len(stranddata['evidenceclip_ids']):
```


Page 112 of 349

```
                strand_data['confidence'] *= 0.5
                strand_data['validation_warning'] = 'Some clip IDs were invalid'

            strand_data['evidence_clips'] = [
                {
                    'id': cid,
                    'start': candidates_by_id[cid].start,
                    'end': candidates_by_id[cid].end
                }
                for cid in valid_clip_ids
            ]

        return result
```

Fallback/Retry Strategy

```
class ResilientLLMScorer:
    def __init__(self, primary_key: str, fallback_key: str = None):
        self.primary = LLMScoringService(primary_key)
        self.fallback = LLMScoringService(fallback_key) if fallback_key else None
```


Page 113 of 349

```
def scorewithretry(self, args, *kwargs) -> DraftReport:
    """Attempt scoring with fallback on failure."""
    try:
        return self.primary.score(args, *kwargs)
    except Exception as e:
        logger.warning(f"Primary LLM failed: {e}")

        if self.fallback:
            return self.fallback.score(args, *kwargs)

        # Return empty draft with error flag
        return DraftReport(
            error=True,
            error_message=str(e)
        )
```

Model Selection Summary

Page 114 of 349

Component	Primary	Fallback	Future
STT	OpenAI Whisper API	AWS Transcribe	Self-hosted Whisper
LLM Scoring	GPT-4o	Claude 3.5 Sonnet	Fine-tuned model
Feature Extraction	Python libraries	-	-
Benchmarking	Custom BDL	-	ML-based classifier

Cost Estimates (per assessment)

Component	Cost
STT (OpenAI)	~$0.006/minute (~$0.01 for 2-min recording)
Feature Extraction	$0 (compute only)
LLM Scoring	~$0.02-0.05 per assessment
Total	**~$0.03-0.06 per assessment**

Quality Assurance

1. **Confidence Thresholds**:

- confidence >= 0.8: High confidence
- 0.5 <= confidence < 0.8: Medium confidence (flag for review)
- confidence < 0.5: Low confidence (require teacher review)

Page 115 of 349

2. **Validation Rules**:

- Clip timestamps must be within video duration
- No duplicate clips across strands
- All scores must map to valid bands

3. **Monitoring**:

- Track LLM token usage
- Log model versions used
- Monitor confidence score distributions
- Alert on high error rates

Backend Structure (Django)

Project Structure

Page 116 of 349

```
server/
├── config/                          # Django project configuration
│   ├── init.py
│   ├── settings/
│   │   ├── init.py
│   │   ├── base.py                 # Shared settings
│   │   ├── development.py          # Local dev settings
│   │   ├── staging.py              # Staging settings
│   │   └── production.py           # Production settings
│   ├── urls.py                     # Root URL configuration
│   ├── wsgi.py                     # WSGI application
│   └── asgi.py                     # ASGI application (WebSocket)
│
├── apps/                           # Django applications
│   ├── init.py
│   ├── users/                      # Authentication & user management
│   │   ├── init.py
│   │   ├── models.py               # User, Teacher, Student models
│   │   ├── serializers.py          # DRF serializers
│   │   ├── views.py                # API views
│   │   ├── urls.py                 # URL routes
│   │   ├── permissions.py          # Custom permissions
│   │   └── tests/
```


Page 117 of 349

```
│   │
│   ├── students/                  # Student & cohort management
│   │   ├── models.py              # Student, Cohort, Enrollment
│   │   ├── serializers.py
│   │   ├── views.py
│   │   └── urls.py
│   │
│   ├── assessments/               # Assessment workflow
│   │   ├── models.py              # Assessment, Recording, Prompt
│   │   ├── serializers.py
│   │   ├── views.py
│   │   ├── urls.py
│   │   └── tasks.py               # Celery tasks for processing
│   │
│   ├── analysis/                  # AI pipeline orchestration
│   │   ├── models.py              # FeatureSignals, EvidenceCandidate
│   │   ├── services/              # External service integrations
│   │   │   ├── __init__.py
│   │   │   ├── stt.py             # STT service interface
│   │   │   ├── llm.py             # LLM scoring service
│   │   │   └── storage.py         # S3 storage service
│   │   ├── pipeline/              # AI pipeline stages
│   │   │   ├── __init__.py
```


Page 118 of 349

```
│   │   │   ├── extract_features.py # Feature extraction
│   │   │   ├── benchmark.py        # BDL interpreter
│   │   │   ├── generate_evidence.py# Evidence candidate generation
│   │   │   └── score_llm.py        # LLM scoring
│   │   ├── tasks.py                # Celery tasks for AI pipeline
│   │   └── views.py                # API for analysis status
│   │
│   ├── reports/                    # Draft & signed reports
│   │   ├── models.py               # DraftReport, SignedReport
│   │   ├── serializers.py
│   │   ├── views.py
│   │   └── exporters.py            # PDF/CSV export logic
│   │
│   ├── benchmarks/                 # BDL management
│   │   ├── models.py               # BenchmarkVersion
│   │   ├── bdl/                    # BDL JSON definitions
│   │   │   ├── v1.0.0/
│   │   │   │   ├── 11-12_explaining.json
│   │   │   │   ├── 11-12_presenting.json
│   │   │   │   └── ...
│   │   └── loader.py               # BDL loading utilities
│   │
│   └── audit/                      # Audit logging
```


Page 119 of 349

```
│       ├── models.py               # AuditLog
│       └── middleware.py           # Auto-logging middleware
│
├── core/                           # Shared utilities
│   ├── init.py
│   ├── models.py                   # Abstract base models
│   ├── permissions.py              # Shared permission classes
│   ├── pagination.py               # Custom pagination
│   ├── filters.py                  # DRF filters
│   └── exceptions.py               # Custom exceptions
│
├── templates/                      # Django templates (admin, emails)
├── static/                         # Static files (admin)
├── media/                          # User uploads (local dev)
├── scripts/                        # Management scripts
│   ├── setup_dev.py
│   └── import_benchmarks.py
├── tests/                          # Integration tests
├── pytest.ini                      # Pytest configuration
├── manage.py                       # Django management
├── Dockerfile                      # Container definition
├── entrypoint.sh                   # Container entrypoint
├── requirements/                   # Dependencies
```


Page 120 of 349

```
│   ├── base.txt
│   ├── development.txt
│   └── production.txt
└── pyproject.toml                  # Modern Python packaging
```

Django Apps & Responsibilities

1. `users` - Authentication & Authorization

Responsibilities:

- User authentication (JWT-based)
- Role management (Admin, Teacher, Student)
- Password management
- User profiles

Page 121 of 349

Key Models:

```
class User(AbstractUser):

    """Extended user model with roles."""

    ROLE_CHOICES = [

        ('admin', 'Administrator'),

        ('teacher', 'Teacher'),

        ('student', 'Student'),

    ]

    role = models.CharField(maxlength=20, choices=ROLECHOICES)

    school = models.ForeignKey(School, ondelete=models.SETNULL, null=True)



    # Teacher-specific

    subject = models.CharField(max_length=100, blank=True)



    # Student-specific

    yeargroup = models.CharField(maxlength=10, blank=True)



class School(models.Model):

    """School/Institution."""

    name = models.CharField(max_length=200)

    identifier = models.CharField(max_length=50, unique=True)
```


Page 122 of 349

```
    region = models.CharField(max_length=50)  # For data residency
```

Key Views:

- `POST /api/auth/login/` - JWT token obtain
- `POST /api/auth/refresh/` - Token refresh
- `POST /api/auth/register/` - User registration (admin only)
- `GET /api/users/me/` - Current user profile
- `PATCH /api/users/me/` - Update profile

---

2. `students` - Student Management

Responsibilities:

- Student profile management
- Cohort/class organization
- Enrollment tracking

Page 123 of 349

Key Models:

```
class Student(models.Model):

    """Student profile."""

    user = models.OneToOneField(User, on_delete=models.CASCADE)

    studentid = models.CharField(maxlength=50, unique=True)

    dateofbirth = models.DateField()

    yeargroup = models.CharField(maxlength=10)  # e.g., "7", "11"

    ageband = models.CharField(maxlength=10)  # e.g., "11-12"



    # Language profile

    EAL = models.BooleanField(default=False)

    firstlanguage = models.CharField(maxlength=50, blank=True)



    # Accommodations

    accommodations = models.JSONField(default=dict, blank=True)



class Cohort(models.Model):

    """Class/group of students."""

    name = models.CharField(max_length=100)

    teacher = models.ForeignKey(User, ondelete=models.CASCADE, limitchoices_to={'role'

    yeargroup = models.CharField(maxlength=10)
```


Page 124 of 349

```
    academicyear = models.CharField(maxlength=9)  # e.g., "2025-2026"
    students = models.ManyToManyField(Student, through='Enrollment')




class Enrollment(models.Model):
    """Student-Cohort relationship."""
    student = models.ForeignKey(Student, on_delete=models.CASCADE)
    cohort = models.ForeignKey(Cohort, on_delete=models.CASCADE)
    enrolledat = models.DateTimeField(autonow_add=True)
```

Key Views:

- `GET /api/students/` - List students (teacher's cohorts only)
- `POST /api/students/` - Create student
- `GET /api/students/{id}/` - Student detail
- `GET /api/cohorts/` - List cohorts
- `GET /api/cohorts/{id}/students/` - Students in cohort

Page 125 of 349

3. `assessments` - Assessment Workflow

Responsibilities:

- Assessment creation and lifecycle
- Video upload handling
- Prompt management
- Processing status tracking

Key Models:

```
class Assessment(models.Model):
    """An oracy assessment."""
    STATUS_CHOICES = [
        ('pending', 'Pending Upload'),
        ('uploading', 'Uploading'),
        ('processing', 'AI Processing'),
        ('draft_ready', 'Draft Ready'),
        ('under_review', 'Under Review'),
        ('signed_off', 'Signed Off'),
        ('error', 'Error'),
```


Page 126 of 349

```
]


MODE_CHOICES = [
    ('presenting', 'Presenting'),
    ('explaining', 'Explaining'),
    ('persuading', 'Persuading'),
]


id = models.UUIDField(primary_key=True, default=uuid.uuid4)
student = models.ForeignKey(Student, on_delete=models.CASCADE)
cohort = models.ForeignKey(Cohort, on_delete=models.CASCADE)


# Assessment config
mode = models.CharField(max_length=20, choices=MODE_CHOICES)
prompt = models.TextField()
time_limit_seconds = models.IntegerField(default=180)  # 3 min default


# Status
status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='pending')
status_message = models.TextField(blank=True)


# Consent
consent_obtained = models.BooleanField(default=False)
```


Page 127 of 349

```
    consent_date = models.DateTimeField(null=True, blank=True)



    # Timestamps

    created_at = models.DateTimeField(auto_now_add=True)

    uploaded_at = models.DateTimeField(null=True, blank=True)

    completed_at = models.DateTimeField(null=True, blank=True)



    class Meta:

        ordering = ['-created_at']



class Recording(models.Model):

    """Video/audio recording for an assessment."""

    assessment = models.OneToOneField(Assessment, on_delete=models.CASCADE)



    # File info

    original_filename = models.CharField(max_length=255)

    s3_key = models.CharField(max_length=500)

    s3_bucket = models.CharField(max_length=100)

    file_size_bytes = models.BigIntegerField()

    duration_seconds = models.FloatField()



    # Processing
```


Page 128 of 349

```
    audio_extracted = models.BooleanField(default=False)

    audios3key = models.CharField(max_length=500, blank=True)



    # Quality

    videoqualityscore = models.FloatField(null=True)  # 0-1

    audioqualityscore = models.FloatField(null=True)  # 0-1



    createdat = models.DateTimeField(autonow_add=True)





class PromptTemplate(models.Model):

    """Reusable prompts for assessments."""

    MODECHOICES = Assessment.MODECHOICES



    name = models.CharField(max_length=100)

    mode = models.CharField(maxlength=20, choices=MODECHOICES)

    text = models.TextField()

    year_groups = models.JSONField(default=list)  # ["7", "8", "9"]

    is_active = models.BooleanField(default=True)
```

Key Views:

Page 129 of 349

- `GET /api/assessments/` - List assessments
- `POST /api/assessments/` - Create assessment
- `GET /api/assessments/{id}/` - Assessment detail
- `POST /api/assessments/{id}/upload-url/` - Get S3 presigned URL for upload
- `POST /api/assessments/{id}/complete-upload/` - Trigger processing after upload
- `GET /api/prompts/` - List prompt templates

Celery Tasks:

```
@app.task(bind=True, max_retries=3)

def processassessment(self, assessmentid: str):

    """Main assessment processing pipeline."""

    assessment = Assessment.objects.get(id=assessment_id)



    try:

        # 1. Extract audio from video

        extractaudiotask.delay(assessment_id)



        # 2. Transcribe (STT)

        transcribetask.delay(assessmentid)
```


Page 130 of 349

```
        # 3. Extract features
        extractfeaturestask.delay(assessment_id)


        # 4. Generate evidence candidates
        generateevidencetask.delay(assessment_id)


        # 5. LLM scoring
        scorewithllmtask.delay(assessmentid)


        # 6. Notify completion
        notifyassessmentcomplete.delay(assessment_id)


    except Exception as exc:
        assessment.status = 'error'
        assessment.status_message = str(exc)
        assessment.save()
        raise self.retry(exc=exc, countdown=60)
```

---

4. `analysis` - AI Pipeline

Page 131 of 349

Responsibilities:

- STT integration
- Feature extraction
- Benchmark application
- Evidence generation
- LLM scoring

Key Models:

```
class Transcript(models.Model):
    """Speech-to-text output."""
    assessment = models.OneToOneField(Assessment, on_delete=models.CASCADE)


    segments = models.JSONField()  # [{start, end, text, words: [...]}]
    full_text = models.TextField()
    language = models.CharField(max_length=10, default='en-GB')
    confidence = models.FloatField()


    # Provider info
    provider = models.CharField(max_length=50)  # 'openai', 'aws'
```


Page 132 of 349

```
    modelversion = models.CharField(maxlength=50)


    createdat = models.DateTimeField(autonow_add=True)


class FeatureSignals(models.Model):
    """Extracted features from audio/transcript."""
    assessment = models.OneToOneField(Assessment, on_delete=models.CASCADE)


    # Physical
    wpm = models.FloatField()
    pause_ratio = models.FloatField()
    filler_frequency = models.FloatField()
    volume_variance = models.FloatField()
    rhythm_stability = models.FloatField()


    # Linguistic
    sentencelengthvariance = models.FloatField()
    vocabulary_diversity = models.FloatField()
    prompt_relevance = models.FloatField()
    clarity_score = models.FloatField()
    register_formality = models.FloatField()


    # Cognitive
```


Page 133 of 349

```
    reason_density = models.FloatField()

    counterpoint_density = models.FloatField()

    logicalconnectordensity = models.FloatField()

    structure_completeness = models.FloatField()



    # Social-emotional

    audiencereferencefrequency = models.FloatField()

    intention_clarity = models.FloatField()

    sentiment_compound = models.FloatField()



    # Quality flag

    quality_flag = models.CharField(

        max_length=20,

        choices=[('good', 'Good'), ('fair', 'Fair'), ('poor', 'Poor')],

        default='good'

    )



    extractedat = models.DateTimeField(autonow_add=True)




class EvidenceCandidate(models.Model):

    """Potential evidence clips."""
```


Page 134 of 349

```
    assessment = models.ForeignKey(Assessment, on_delete=models.CASCADE)



    candidateid = models.CharField(maxlength=20)  # "clip0", "clip1", etc.

    start_time = models.FloatField()  # seconds

    end_time = models.FloatField()

    type = models.CharField(maxlength=50)  # 'transcriptboundary', 'reason_dense', etc

    summary = models.TextField()

    features = models.JSONField()



    class Meta:

        ordering = ['start_time']
```

Service Interfaces:

```
# apps/analysis/services/stt.py

from typing import Protocol

from dataclasses import dataclass



@dataclass

class TranscriptSegment:

    start: float

    end: float
```


Page 135 of 349

```
    text: str
    words: List[Word]




class STTService(Protocol):
    """Abstract STT service interface."""

    def transcribe(self, audio_path: str) -> List[TranscriptSegment]:
        ...

    def supports_streaming(self) -> bool:
        ...




class WhisperService:
    """OpenAI Whisper implementation."""

    def init(self, api_key: str):
        self.client = OpenAI(apikey=apikey)

    def transcribe(self, audio_path: str) -> List[TranscriptSegment]:
        # Implementation...
        pass
```


Page 136 of 349

```
class AWSTranscribeService:
    """AWS Transcribe implementation."""

    def transcribe(self, audio_path: str) -> List[TranscriptSegment]:
        # Implementation...
        pass
```

---

5. `reports` - Report Management

Responsibilities:

- Draft report storage
- Teacher review workflow
- Signed report archival
- Export generation (PDF/CSV)

Page 137 of 349

Key Models:

```
class DraftReport(models.Model):

    """AI-generated draft report."""

    assessment = models.OneToOneField(Assessment, on_delete=models.CASCADE)



    # Strand scores

    physicalscore = models.JSONField()  # {band, confidence, evidenceclips, justificat

    linguistic_score = models.JSONField()

    cognitive_score = models.JSONField()

    socialemotionalscore = models.JSONField()



    # Feedback

    feedback = models.JSONField()  # {strengths: [], next_steps: [], goals: []}



    # Metadata

    aimodel = models.CharField(maxlength=50)  # 'gpt-4o'

    generatedat = models.DateTimeField(autonow_add=True)



    # Status

    is_reviewed = models.BooleanField(default=False)
```


Page 138 of 349

```
class SignedReport(models.Model):

    """Teacher-signed final report."""

    assessment = models.OneToOneField(Assessment, on_delete=models.CASCADE)

    draft = models.OneToOneField(DraftReport, ondelete=models.SETNULL, null=True)



    # Final scores (may differ from draft)

    physical_score = models.JSONField()

    linguistic_score = models.JSONField()

    cognitive_score = models.JSONField()

    socialemotionalscore = models.JSONField()



    # Final feedback

    feedback = models.JSONField()

    teacher_notes = models.TextField(blank=True)



    # Signing

    signedby = models.ForeignKey(User, ondelete=models.SET_NULL, null=True)

    signedat = models.DateTimeField(autonow_add=True)





class AuditLog(models.Model):

    """Track all changes to reports."""

    ACTION_CHOICES = [
```


Page 139 of 349

```
        ('score_changed', 'Score Changed'),
        ('clip_changed', 'Evidence Clip Changed'),
        ('feedback_edited', 'Feedback Edited'),
        ('signed', 'Report Signed'),
    ]

    report = models.ForeignKey(DraftReport, on_delete=models.CASCADE)
    user = models.ForeignKey(User, ondelete=models.SETNULL, null=True)
    action = models.CharField(maxlength=20, choices=ACTIONCHOICES)
    field = models.CharField(max_length=50)
    old_value = models.JSONField(null=True)
    new_value = models.JSONField(null=True)
    timestamp = models.DateTimeField(autonowadd=True)
```

Key Views:

- `GET /api/reports/draft/{assessment_id}/` - Get draft report
- `POST /api/reports/draft/{assessment_id}/edit/` - Edit draft
- `POST /api/reports/{assessment_id}/sign/` - Sign off on report
- `GET /api/reports/{assessment_id}/export/pdf/` - Generate PDF

Page 140 of 349

PDF Export:

```
from reportlab.pdfgen import canvas

from reportlab.lib.pagesizes import A4



class PDFExporter:

    """Generate inspection-grade PDF reports."""



    def export(self, report: SignedReport) -> bytes:

        buffer = BytesIO()

        c = canvas.Canvas(buffer, pagesize=A4)



        # Header

        c.setFont("Helvetica-Bold", 18)

        c.drawString(50, 800, "Oracy Assessment Report")



        # Student info

        c.setFont("Helvetica", 12)

        student = report.assessment.student

        c.drawString(50, 760, f"Student: {student.user.getfullname()}")

        c.drawString(50, 740, f"Year Group: {student.year_group}")

        c.drawString(50, 720, f"Date: {report.signed_at.strftime('%d %B %Y')}")
```


Page 141 of 349

```
    # Scores
    y = 680
    for strand, score_data in [
        ('Physical', report.physical_score),
        ('Linguistic', report.linguistic_score),
        ('Cognitive', report.cognitive_score),
        ('Social-Emotional', report.socialemotionalscore),
    ]:
        c.setFont("Helvetica-Bold", 14)
        c.drawString(50, y, f"{strand}: {score_data['band'].title()}")
        y -= 20

        # Evidence clips
        c.setFont("Helvetica", 10)
        for clip in scoredata['evidenceclips'][:3]:  # Top 3 clips
            c.drawString(70, y, f"• [{clip['start']:.1f}s - {clip['end']:.1f}s]")
            y -= 15

        y -= 10

    c.save()
    return buffer.getvalue()
```


Page 142 of 349

6. `benchmarks` - BDL Management

Responsibilities:

- Benchmark version storage
- BDL loading and validation
- Version management

Key Models:

```
class BenchmarkVersion(models.Model):

    """A versioned benchmark definition."""

    version = models.CharField(max_length=20)

    ageband = models.CharField(maxlength=10)  # "11-12"

    mode = models.CharField(max_length=20)  # "explaining"



    definition = models.JSONField()  # Full BDL
```


Page 143 of 349

```
    is_active = models.BooleanField(default=True)

    created_at = models.DateTimeField(auto_now_add=True)



    class Meta:

        unique_together = ['version', 'age_band', 'mode']

        ordering = ['-created_at']



class BenchmarkLoader:

    """Load BDL from JSON files into database."""



    def load_from_directory(self, directory: Path):

        """Load all BDL files from a directory."""

        for file_path in directory.glob('*/.json'):

            with open(file_path) as f:

                definition = json.load(f)



            BenchmarkVersion.objects.get_or_create(

                version=definition['version'],

                age_band=definition['age_band'],

                mode=definition['mode'],

                defaults={'definition': definition}
```


Page 144 of 349

```
            )
```

API Design

REST Endpoints Summary

Endpoint	Method	Description
`/api/auth/login/`	POST	Obtain JWT tokens
`/api/auth/refresh/`	POST	Refresh access token
`/api/users/me/`	GET/PATCH	Current user profile
`/api/students/`	GET/POST	List/create students
`/api/students/{id}/`	GET/PUT/DELETE	Student detail
`/api/cohorts/`	GET/POST	List/create cohorts
`/api/cohorts/{id}/students/`	GET	Students in cohort
`/api/assessments/`	GET/POST	List/create assessments
`/api/assessments/{id}/`	GET	Assessment detail
`/api/assessments/{id}/upload-url/`	POST	Get S3 presigned URL
`/api/assessments/{id}/status/`	GET	Processing status
`/api/reports/draft/{id}/`	GET	Get draft report
`/api/reports/draft/{id}/edit/`	POST	Edit draft
`/api/reports/{id}/sign/`	POST	Sign report
`/api/reports/{id}/export/pdf/`	GET	Download PDF
`/api/benchmarks/`	GET	List active benchmarks
`/api/audit/{report_id}/`	GET	Audit trail

Page 145 of 349

WebSocket Endpoints

Endpoint	Purpose
`/ws/assessments/{id}/`	Real-time processing updates

```
# consumers.py

from channels.generic.websocket import AsyncJsonWebsocketConsumer



class AssessmentConsumer(AsyncJsonWebsocketConsumer):

    async def connect(self):

        self.assessmentid = self.scope['urlroute']['kwargs']['id']

        self.groupname = f"assessment{self.assessment_id}"



        await self.channellayer.groupadd(

            self.group_name,

            self.channel_name

        )

        await self.accept()



    async def assessment_update(self, event):
```


Page 146 of 349

```
        await self.send_json({
            'type': 'status_update',
            'status': event['status'],
            'message': event.get('message', '')
        })
```

Settings Configuration

base.py

```
import os
from pathlib import Path


BASE_DIR = Path(file).resolve().parent.parent.parent
```


Page 147 of 349

```
INSTALLED_APPS = [
    # Django core
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    # Third-party
    'rest_framework',
    'corsheaders',
    'djangocelerybeat',
    'channels',

    # Local apps
    'apps.users',
    'apps.students',
    'apps.assessments',
    'apps.analysis',
    'apps.reports',
    'apps.benchmarks',
    'apps.audit',
```


Page 148 of 349

```
]
```

Database

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': os.getenv('DB_NAME', 'oracy'),
        'USER': os.getenv('DB_USER', 'oracy'),
        'PASSWORD': os.getenv('DB_PASSWORD', 'oracy'),
        'HOST': os.getenv('DB_HOST', 'localhost'),
        'PORT': os.getenv('DB_PORT', '5432'),
    }
}
```


Page 149 of 349

Redis/Cache

```
CACHES = {
    'default': {
        'BACKEND': 'django_redis.cache.RedisCache',
        'LOCATION': os.getenv('REDIS_URL', 'redis://localhost:6379/0'),
    }
}
```


Page 150 of 349

Celery

```
CELERY_BROKER_URL = os.getenv('REDIS_URL', 'redis://localhost:6379/0')

CELERY_RESULT_BACKEND = os.getenv('REDIS_URL', 'redis://localhost:6379/0')
```


Page 151 of 349

REST Framework

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],
    'DEFAULT_PAGINATION_CLASS': 'core.pagination.StandardResultsSetPagination',
    'DEFAULT_FILTER_BACKENDS': [
        'django_filters.rest_framework.DjangoFilterBackend',
    ],
}
```


Page 152 of 349

Channels (WebSocket)

```
ASGI_APPLICATION = 'config.asgi.application'

CHANNEL_LAYERS = {

    'default': {

        'BACKEND': 'channels_redis.core.RedisChannelLayer',

        'CONFIG': {

            'hosts': [os.getenv('REDIS_URL', 'redis://localhost:6379/0')],

        },

    },

}
```


Page 153 of 349

File Storage (S3)

```
DEFAULTFILESTORAGE = 'storages.backends.s3boto3.S3Boto3Storage'

AWSSTORAGEBUCKETNAME = os.getenv('AWSSTORAGEBUCKETNAME')

AWSS3REGIONNAME = os.getenv('AWSREGION', 'ap-southeast-1')

AWSS3SIGNATURE_VERSION = 's3v4'

AWSS3FILE_OVERWRITE = False

AWSDEFAULTACL = 'private'

AWSS3VERIFY = True
```


Page 154 of 349

```
# AI Services


OPENAIAPIKEY = os.getenv('OPENAIAPIKEY')

ANTHROPICAPIKEY = os.getenv('ANTHROPICAPIKEY')
```

Testing Strategy

Test Structure

```
tests/

├── conftest.py                 # Pytest fixtures

├── factories.py                # Model factories (factory-boy)

├── unit/

│   ├── test_users.py

│   ├── test_assessments.py

│   └── test_analysis.py

├── integration/
```


Page 155 of 349

```
│   ├── test_api.py
│   └── test_pipeline.py
└── e2e/
    └── test_full_workflow.py
```

Key Fixtures

```
# conftest.py
import pytest
from apps.users.models import User


@pytest.fixture
def teacher_user(db):
    return User.objects.create_user(
        username='teacher@school.edu',
        email='teacher@school.edu',
        password='testpass123',
        role='teacher'
    )
```


Page 156 of 349

```
@pytest.fixture

def apiclient(teacheruser):

    from rest_framework.test import APIClient

    client = APIClient()

    client.forceauthenticate(user=teacheruser)

    return client




@pytest.fixture

def mocksttservice(monkeypatch):

    """Mock STT service for tests."""

    class MockSTT:

        def transcribe(self, audio_path):

            return [

                TranscriptSegment(

                    start=0.0,

                    end=5.0,

                    text="Test transcript",

                    words=[...]

                )

            ]
```


Page 157 of 349

```
    monkeypatch.setattr(
        'apps.analysis.services.stt.WhisperService',
        MockSTT
    )
```

Frontend Structure (React)

Project Structure

```
client/
├── public/
│   ├── index.html
│   ├── favicon.ico
│   └── manifest.json
│
├── src/
│   ├── main.tsx                    # Entry point
│   ├── App.tsx                     # Root component
│   ├── routes.tsx                  # Route definitions
│   |
```


Page 158 of 349

```
│   ├── api/                        # API client
│   │   ├── client.ts               # Axios instance
│   │   ├── auth.ts                 # Auth endpoints
│   │   ├── assessments.ts          # Assessment endpoints
│   │   ├── students.ts             # Student endpoints
│   │   ├── reports.ts              # Report endpoints
│   │   └── types.ts                # API type definitions
│   │
│   ├── components/                 # Reusable components
│   │   ├── ui/                     # Base UI components
│   │   │   ├── Button.tsx
│   │   │   ├── Card.tsx
│   │   │   ├── Input.tsx
│   │   │   ├── Select.tsx
│   │   │   ├── Modal.tsx
│   │   │   ├── Loading.tsx
│   │   │   ├── ErrorBoundary.tsx
│   │   │   └── index.ts
│   │   │
│   │   ├── layout/                 # Layout components
│   │   │   ├── MainLayout.tsx
│   │   │   ├── Sidebar.tsx
│   │   │   ├── Header.tsx
```


Page 159 of 349

```
│   │   │   ├── Footer.tsx
│   │   │   └── index.ts
│   │   │
│   │   ├── video/                  # Video-related components
│   │   │   ├── VideoRecorder.tsx   # WebRTC recording
│   │   │   ├── VideoPlayer.tsx     # Video playback
│   │   │   ├── VideoTimeline.tsx   # Timestamp navigation
│   │   │   ├── ClipSelector.tsx    # Evidence clip selection
│   │   │   └── index.ts
│   │   │
│   │   ├── assessment/             # Assessment components
│   │   │   ├── AssessmentCard.tsx
│   │   │   ├── AssessmentList.tsx
│   │   │   ├── AssessmentForm.tsx
│   │   │   ├── StatusBadge.tsx
│   │   │   └── index.ts
│   │   │
│   │   ├── report/                 # Report components
│   │   │   ├── ReportViewer.tsx
│   │   │   ├── StrandScore.tsx
│   │   │   ├── EvidenceClips.tsx
│   │   │   ├── FeedbackSection.tsx
│   │   │   ├── ScoreEditor.tsx
```


Page 160 of 349

```
│   │   │   └── index.ts
│   │   │
│   │   └── dashboard/              # Dashboard components
│   │       ├── StatsCard.tsx
│   │       ├── RecentAssessments.tsx
│   │       ├── CohortSelector.tsx
│   │       └── index.ts
│   │
│   ├── hooks/                      # Custom React hooks
│   │   ├── useAuth.ts              # Authentication state
│   │   ├── useApi.ts               # API request wrapper
│   │   ├── useVideoRecorder.ts     # Recording logic
│   │   ├── useAssessment.ts        # Assessment data
│   │   ├── useReport.ts            # Report data
│   │   └── index.ts
│   │
│   ├── pages/                      # Page components
│   │   ├── auth/
│   │   │   ├── LoginPage.tsx
│   │   │   └── index.ts
│   │   │
│   │   ├── dashboard/
│   │   │   ├── DashboardPage.tsx
```


Page 161 of 349

```
│   │   │   └── index.ts
│   │   │
│   │   ├── students/
│   │   │   ├── StudentsListPage.tsx
│   │   │   ├── StudentDetailPage.tsx
│   │   │   └── index.ts
│   │   │
│   │   ├── assessments/
│   │   │   ├── AssessmentListPage.tsx
│   │   │   ├── AssessmentCreatePage.tsx
│   │   │   ├── AssessmentRecordPage.tsx
│   │   │   ├── AssessmentReviewPage.tsx
│   │   │   └── index.ts
│   │   │
│   │   ├── reports/
│   │   │   ├── ReportViewPage.tsx
│   │   │   ├── ReportEditPage.tsx
│   │   │   └── index.ts
│   │   │
│   │   └── settings/
│   │       ├── SettingsPage.tsx
│   │       └── index.ts
│   │
```


Page 162 of 349

```
│   ├── stores/                    # Zustand stores
│   │   ├── authStore.ts           # Auth state
│   │   ├── assessmentStore.ts     # Assessment state
│   │   ├── uiStore.ts             # UI state (modals, toasts)
│   │   └── index.ts
│   │
│   ├── utils/                     # Utility functions
│   │   ├── formatters.ts          # Date, time formatters
│   │   ├── validators.ts          # Form validation
│   │   ├── constants.ts           # App constants
│   │   └── index.ts
│   │
│   ├── types/                     # TypeScript types
│   │   ├── auth.ts
│   │   ├── user.ts
│   │   ├── student.ts
│   │   ├── assessment.ts
│   │   ├── report.ts
│   │   └── index.ts
│   │
│   └── styles/                    # Global styles
│       ├── index.css              # Tailwind imports
│       └── variables.css          # CSS variables
```


Page 163 of 349

```
│
├── tests/                          # Test files
│   ├── unit/
│   ├── integration/
│   └── e2e/
│
├── .env                            # Environment variables
├── .env.example
├── index.html
├── package.json
├── tsconfig.json
├── vite.config.ts
├── tailwind.config.js
├── postcss.config.js
├── eslint.config.js
└── playwright.config.ts
```

Component Architecture

Page 164 of 349

Core UI Components

Button.tsx

```
import React from 'react';

import { cva, type VariantProps } from 'class-variance-authority';



const buttonVariants = cva(

  'inline-flex items-center justify-center rounded-md text-sm font-medium transition-c

  {

    variants: {

      variant: {

        primary: 'bg-blue-600 text-white hover:bg-blue-700',

        secondary: 'bg-gray-200 text-gray-900 hover:bg-gray-300',

        danger: 'bg-red-600 text-white hover:bg-red-700',

        ghost: 'hover:bg-gray-100',

      },

      size: {

        sm: 'h-8 px-3',

        md: 'h-10 px-4',
```


Page 165 of 349

```
        lg: 'h-12 px-6',
      },
    },
    defaultVariants: {
      variant: 'primary',
      size: 'md',
    },
  }
);




interface ButtonProps
  extends React.ButtonHTMLAttributes,
    VariantProps {
  isLoading?: boolean;
}




export const Button: React.FC = ({
  children,
  variant,
  size,
  isLoading,
```


Page 166 of 349

```
  disabled,
  ...props
}) => {
  return (

      className={buttonVariants({ variant, size })}
      disabled={disabled || isLoading}
      {...props}
    >
      {isLoading && }
      {children}


  );
};
```

Video Components

VideoRecorder.tsx

```
import React, { useRef, useState, useCallback } from 'react';
```


Page 167 of 349

```
interface VideoRecorderProps {
  onRecordingComplete: (blob: Blob) => void;
  maxDuration?: number; // seconds
  countdown?: number; // countdown before recording
}




export const VideoRecorder: React.FC = ({
  onRecordingComplete,
  maxDuration = 180,
  countdown = 3,
}) => {
  const videoRef = useRef(null);
  const mediaRecorderRef = useRef(null);
  const chunksRef = useRef([]);


  const [isRecording, setIsRecording] = useState(false);
  const [isCountingDown, setIsCountingDown] = useState(false);
  const [countdownValue, setCountdownValue] = useState(countdown);
  const [recordingTime, setRecordingTime] = useState(0);
  const [stream, setStream] = useState(null);
  const [error, setError] = useState(null);
```


Page 168 of 349

```
const initializeCamera = useCallback(async () => {
    try {
      const mediaStream = await navigator.mediaDevices.getUserMedia({
        video: { width: 1280, height: 720, facingMode: 'user' },
        audio: { echoCancellation: true, noiseSuppression: true },
      });

      setStream(mediaStream);
      if (videoRef.current) {
        videoRef.current.srcObject = mediaStream;
      }
    } catch (err) {
      setError('Unable to access camera. Please check permissions.');
    }
  }, []);


const startCountdown = useCallback(() => {
    setIsCountingDown(true);
    setCountdownValue(countdown);
```


Page 169 of 349

```
    const timer = setInterval(() => {
      setCountdownValue((prev) => {
        if (prev <= 1) {
          clearInterval(timer);
          setIsCountingDown(false);
          startRecording();
          return 0;
        }
        return prev - 1;
      });
    }, 1000);
  }, [countdown]);




const startRecording = useCallback(() => {
    if (!stream) return;


    const mediaRecorder = new MediaRecorder(stream, {
      mimeType: 'video/webm;codecs=vp9,opus',
    });


    mediaRecorderRef.current = mediaRecorder;
```


Page 170 of 349

```
    chunksRef.current = [];


    mediaRecorder.ondataavailable = (event) => {
      if (event.data.size > 0) {
        chunksRef.current.push(event.data);
      }
    };


    mediaRecorder.onstop = () => {
      const blob = new Blob(chunksRef.current, { type: 'video/webm' });
      onRecordingComplete(blob);
    };


    mediaRecorder.start(1000); // Collect data every second
    setIsRecording(true);


    // Timer for recording duration
    const timer = setInterval(() => {
      setRecordingTime((prev) => {
        if (prev >= maxDuration) {
          stopRecording();
          clearInterval(timer);
          return prev;
```


Page 171 of 349

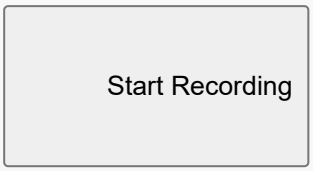
```
        }
        return prev + 1;
      });
    }, 1000);
  }, [stream, maxDuration, onRecordingComplete]);




const stopRecording = useCallback(() => {
    if (mediaRecorderRef.current && isRecording) {
      mediaRecorderRef.current.stop();
      setIsRecording(false);


      // Stop all tracks
      stream?.getTracks().forEach(track => track.stop());
    }
  }, [isRecording, stream]);




return (




        ref={videoRef}
```


Page 172 of 349

```
        autoPlay

        muted

        playsInline

        className="w-full aspect-video"

      />



      {isCountingDown && (



          {countdownValue}



      )}



      {isRecording && (



                    {Math.floor(recordingTime / 60)}:{String(recordingTime % 60).padStart(2, '



      )}
```


Page 173 of 349

```
        {!isRecording && !isCountingDown && (
```




```
        )}


        {isRecording && (
```

```
        )}




      {error && (




{error}




      )}
```


Page 174 of 349

```
  );

};
```

VideoPlayer with Timeline

```
import React, { useRef, useState, useCallback } from 'react';




interface Clip {

  id: string;

  start: number;

  end: number;

  type: string;

}




interface VideoPlayerProps {

  src: string;

  clips?: Clip[];

  onClipClick?: (clip: Clip) => void;

  activeClipId?: string;

}
```


Page 175 of 349

```
export const VideoPlayer: React.FC = ({
  src,
  clips = [],
  onClipClick,
  activeClipId,
}) => {
  const videoRef = useRef(null);
  const [currentTime, setCurrentTime] = useState(0);
  const [duration, setDuration] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);



const handleTimeUpdate = useCallback(() => {
    if (videoRef.current) {
      setCurrentTime(videoRef.current.currentTime);
    }
  }, []);



const handleLoadedMetadata = useCallback(() => {
```


Page 176 of 349

```
    if (videoRef.current) {
      setDuration(videoRef.current.duration);
    }
  }, []);




const seekTo = useCallback((time: number) => {
    if (videoRef.current) {
      videoRef.current.currentTime = time;
    }
  }, []);




const playClip = useCallback((clip: Clip) => {
    if (videoRef.current) {
      videoRef.current.currentTime = clip.start;
      videoRef.current.play();
      setIsPlaying(true);


      // Auto-pause at end of clip
      const checkEnd = () => {
        if (videoRef.current && videoRef.current.currentTime >= clip.end) {
          videoRef.current.pause();
```


Page 177 of 349

```
          setIsPlaying(false);
          videoRef.current.removeEventListener('timeupdate', checkEnd);
        }
      };

      videoRef.current.addEventListener('timeupdate', checkEnd);
    }

    onClipClick?.(clip);
  }, [onClipClick]);

return (

        ref={videoRef}
        src={src}
        className="w-full rounded-lg"
        onTimeUpdate={handleTimeUpdate}
        onLoadedMetadata={handleLoadedMetadata}
        onPlay={() => setIsPlaying(true)}
        onPause={() => setIsPlaying(false)}
```


Page 178 of 349

```
        controls

      />



      {/ Timeline with clips /}




        {/ Progress bar /}


          className="absolute h-full bg-blue-200"

          style={{ width:
${(currentTime / duration) * 100}%
 }}

        />



        {/ Clip markers /}

        {clips.map((clip) => (


            key={clip.id}

            onClick={() => playClip(clip)}

            className={
absolute h-full transition-colors ${

             clip.id === activeClipId

               ? 'bg-green-500'

               : 'bg-blue-500 hover:bg-blue-600'

           }
}

            style={{
```


Page 179 of 349

```
                left:
${(clip.start / duration) * 100}%
,

                width:
${((clip.end - clip.start) / duration) * 100}%
,

              }}

              title={
${clip.type}: ${clip.start.toFixed(1)}s - ${clip.end.toFixed(1)}s
}

            />

          ))}



          {/ Time markers /}




            0:00

            {Math.floor(duration / 60)}:{String(Math.floor(duration % 60)).padStart(2, '
```

```
        {/ Clip list /}

        {clips.length > 0 && (
```


Page 180 of 349

```
Evidence Clips




            {clips.map((clip) => (


                key={clip.id}

                onClick={() => playClip(clip)}

                className={
px-3 py-1 text-sm rounded-full transition-colors ${

                clip.id === activeClipId

                  ? 'bg-green-100 text-green-800 border border-green-300'

                  : 'bg-gray-100 text-gray-700 hover:bg-gray-200'

               }
}

               >

                {clip.start.toFixed(1)}s - {clip.end.toFixed(1)}s



            ))}




      )}



  );
```


Page 181 of 349

```
};
```

State Management

Auth Store (Zustand)

```
// stores/authStore.ts

import { create } from 'zustand';

import { persist } from 'zustand/middleware';



interface User {

  id: string;

  email: string;

  firstName: string;

  lastName: string;

  role: 'admin' | 'teacher' | 'student';

}
```


Page 182 of 349

```
interface AuthState {
  user: User | null;
  accessToken: string | null;
  refreshToken: string | null;
  isAuthenticated: boolean;

  // Actions
  setAuth: (user: User, accessToken: string, refreshToken: string) => void;
  logout: () => void;
  updateUser: (user: Partial) => void;
}


export const useAuthStore = create()(
  persist(
    (set) => ({
      user: null,
      accessToken: null,
      refreshToken: null,
      isAuthenticated: false,
```


Page 183 of 349

```
    setAuth: (user, accessToken, refreshToken) =>
      set({
        user,
        accessToken,
        refreshToken,
        isAuthenticated: true,
      }),


    logout: () =>
      set({
        user: null,
        accessToken: null,
        refreshToken: null,
        isAuthenticated: false,
      }),


    updateUser: (updates) =>
      set((state) => ({
        user: state.user ? { ...state.user, ...updates } : null,
      })),
  }),
  {
    name: 'auth-storage',
```


Page 184 of 349

```
      partialize: (state) => ({
        user: state.user,
        accessToken: state.accessToken,
        refreshToken: state.refreshToken,
        isAuthenticated: state.isAuthenticated,
      }),
    }
  )
);
```

Assessment Store

```
// stores/assessmentStore.ts
import { create } from 'zustand';



interface Assessment {
  id: string;
  studentId: string;
  mode: 'presenting' | 'explaining' | 'persuading';
  prompt: string;
```


Page 185 of 349

```
  status: string;

  createdAt: string;

}




interface AssessmentState {

  assessments: Assessment[];

  currentAssessment: Assessment | null;

  isLoading: boolean;

  error: string | null;



  // Actions

  setAssessments: (assessments: Assessment[]) => void;

  setCurrentAssessment: (assessment: Assessment | null) => void;

  addAssessment: (assessment: Assessment) => void;

  updateAssessment: (id: string, updates: Partial) => void;

  setLoading: (isLoading: boolean) => void;

  setError: (error: string | null) => void;

}




export const useAssessmentStore = create((set) => ({

  assessments: [],
```


Page 186 of 349

```
  currentAssessment: null,
  isLoading: false,
  error: null,


  setAssessments: (assessments) => set({ assessments }),
  setCurrentAssessment: (assessment) => set({ currentAssessment: assessment }),
  addAssessment: (assessment) =>
    set((state) => ({
      assessments: [assessment, ...state.assessments],
    })),
  updateAssessment: (id, updates) =>
    set((state) => ({
      assessments: state.assessments.map((a) =>
        a.id === id ? { ...a, ...updates } : a
      ),
    })),
  setLoading: (isLoading) => set({ isLoading }),
  setError: (error) => set({ error }),
}));
```


Page 187 of 349

API Client

Axios Instance

```
// api/client.ts

import axios from 'axios';

import { useAuthStore } from '@/stores/authStore';




const apiClient = axiosCreate({

  baseURL: import.meta.env.VITEAPIURL || 'http://localhost:8000/api',

  headers: {

    'Content-Type': 'application/json',

  },

});




// Request interceptor - add auth token

apiClient.interceptors.request.use((config) => {

  const token = useAuthStore.getState().accessToken;

  if (token) {
```


Page 188 of 349

```
    config.headers.Authorization =
Bearer ${token}
;

  }

  return config;

});




// Response interceptor - handle token refresh

apiClient.interceptors.response.use(

  (response) => response,

  async (error) => {

    const originalRequest = error.config;



    if (error.response?.status === 401 && !originalRequest._retry) {

      originalRequest._retry = true;



      const refreshToken = useAuthStore.getState().refreshToken;



      try {

        const response = await axios.post(


${import.meta.env.VITEAPIURL}/auth/refresh/
,
```


Page 189 of 349

```
          { refresh: refreshToken }

        );



        const { access } = response.data;

        useAuthStore.getState().accessToken = access;



        originalRequest.headers.Authorization =
Bearer ${access}
;

        return apiClient(originalRequest);

      } catch (refreshError) {

        useAuthStore.getset().logout();

        window.location.href = '/login';

        return Promise.reject(refreshError);

      }

    }



    return Promise.reject(error);

  }

);





export default apiClient;
```


Page 190 of 349

API Hooks

```
// hooks/useApi.ts

import { useState, useCallback } from 'react';

import apiClient from '@/api/client';



interface UseApiOptions {

  onSuccess?: (data: any) => void;

  onError?: (error: Error) => void;

}





export function useApi(options: UseApiOptions = {}) {

  const [data, setData] = useState(null);

  const [isLoading, setIsLoading] = useState(false);

  const [error, setError] = useState(null);





const execute = useCallback(

    async (method: string, url: string, body?: any) => {

      setIsLoading(true);
```


Page 191 of 349

```
      setError(null);


      try {
        const response = await apiClient.request({
          method,
          url,
          data: body,
        });


        setData(response.data);
        options.onSuccess?.(response.data);
        return response.data;
      } catch (err) {
        const error = err instanceof Error ? err : new Error(String(err));
        setError(error);
        options.onError?.(error);
        throw error;
      } finally {
        setIsLoading(false);
      }
    },
    [options]
  );
```


Page 192 of 349

```
return { data, isLoading, error, execute };
}



// Specialized hooks
export function useGet(url: string) {
  const api = useApi();

  const get = useCallback(() => {
    return api.execute('GET', url);
  }, [url]);

  return { ...api, get };
}



export function usePost(url: string, options?: UseApiOptions) {
  const api = useApi(options);

  const post = useCallback(
```


Page 193 of 349

```
    (body: any) => {
      return api.execute('POST', url, body);
    },
    [url]
  );

  return { ...api, post };
}
```

Routing

```
// routes.tsx
import { createBrowserRouter, Navigate } from 'react-router-dom';
import { MainLayout } from '@/components/layout';
import { ProtectedRoute } from '@/components/auth';

// Pages
```


Page 194 of 349

```
import { LoginPage } from '@/pages/auth';

import { DashboardPage } from '@/pages/dashboard';

import { StudentsListPage, StudentDetailPage } from '@/pages/students';

import {

  AssessmentListPage,

  AssessmentCreatePage,

  AssessmentRecordPage,

  AssessmentReviewPage,

} from '@/pages/assessments';

import { ReportViewPage, ReportEditPage } from '@/pages/reports';





export const router = createBrowserRouter([

  {

    path: '/login',

    element: ,

  },

  {

    path: '/',

    element: (
```


Page 195 of 349

```
    ),
    children: [
      {
        index: true,
        element: ,
      },
      {
        path: 'dashboard',
        element: ,
      },
      {
        path: 'students',
        children: [
          { index: true, element:  },
          { path: ':id', element:  },
        ],
      },
      {
        path: 'assessments',
        children: [
          { index: true, element:  },
          { path: 'new', element:  },
          { path: ':id/record', element:  },
```


Page 196 of 349

```
          { path: ':id/review', element:  },
        ],
      },
      {
        path: 'reports',
        children: [
          { path: ':assessmentId', element:  },
          { path: ':assessmentId/edit', element:  },
        ],
      },
    ],
  },
]);
```

Key Features Implementation

Real-time Status Updates (WebSocket)

Page 197 of 349

```
// hooks/useAssessmentStatus.ts

import { useEffect, useRef } from 'react';

import { useAssessmentStore } from '@/stores/assessmentStore';




export function useAssessmentStatus(assessmentId: string) {

  const wsRef = useRef(null);

  const updateAssessment = useAssessmentStore((state) => state.updateAssessment);





useEffect(() => {

    const wsUrl =
${import.meta.env.VITEWSURL}/assessments/${assessmentId}/
;

    const ws = new WebSocket(wsUrl);

    wsRef.current = ws;





ws.onmessage = (event) => {

      const data = JSON.parse(event.data);



      if (data.type === 'status_update') {

        updateAssessment(assessmentId, {
```


Page 198 of 349

```
          status: data.status,

          statusMessage: data.message,

        });

      }

    };




ws.onerror = (error) => {

      console.error('WebSocket error:', error);

    };




return () => {

      ws.close();

    };

  }, [assessmentId, updateAssessment]);

}
```

Video Upload with Progress

Page 199 of 349

```
// components/video/VideoUploader.tsx

import React, { useState, useCallback } from 'react';

import axios from 'axios';



interface VideoUploaderProps {

  assessmentId: string;

  onUploadComplete: () => void;

}





export const VideoUploader: React.FC = ({

  assessmentId,

  onUploadComplete,

}) => {

  const [uploadProgress, setUploadProgress] = useState(0);

  const [isUploading, setIsUploading] = useState(false);





const uploadVideo = useCallback(

    async (file: File) => {

      setIsUploading(true);

      setUploadProgress(0);
```


Page 200 of 349

```
try {
        // 1. Get presigned URL from backend
        const { data } = await axios.post(
/api/assessments/${assessmentId}/upload-url/
,
          {
            filename: file.name,
            contentType: file.type,
            fileSize: file.size,
          }
        );




// 2. Upload directly to S3
        await axios.put(data.uploadUrl, file, {
          headers: {
            'Content-Type': file.type,
          },
          onUploadProgress: (progressEvent) => {
            const progress = Math.round(
              (progressEvent.loaded * 100) / (progressEvent.total || 1)
```


Page 201 of 349

```
            );
            setUploadProgress(progress);
          },
        });




// 3. Notify backend upload is complete
        await axios.post(
/api/assessments/${assessmentId}/complete-upload/
,
          {
            s3Key: data.s3Key,
          }
        );




onUploadComplete();
      } catch (error) {
        console.error('Upload failed:', error);
        // Show error toast
      } finally {
        setIsUploading(false);
```


Page 202 of 349

```
      }

    },

    [assessmentId, onUploadComplete]

  );




return (




        type="file"

        accept="video/*"

        onChange={(e) => e.target.files?.[0] && uploadVideo(e.target.files[0])}

        disabled={isUploading}

      />



      {isUploading && (









              className="h-full bg-blue-600 transition-all"

              style={{ width:
${uploadProgress}%
 }}
```


Page 203 of 349

```
            />




{uploadProgress}% uploaded




      )}



  );

};
```

Styling (Tailwind)

tailwind.config.js

```
/* @type {import('tailwindcss').Config} /

export default {
```


Page 204 of 349

```
content: ['./index.html', './src/*/.{js,ts,jsx,tsx}'],

theme: {

  extend: {

    colors: {

      primary: {

        50: '#eff6ff',

        100: '#dbeafe',

        500: '#3b82f6',

        600: '#2563eb',

        700: '#1d4ed8',

      },

      strand: {

        physical: '#ef4444',      // Red

        linguistic: '#3b82f6',    // Blue

        cognitive: '#22c55e',     // Green

        social: '#a855f7',        // Purple

      },

      band: {

        emerging: '#f97316',      // Orange

        expected: '#22c55e',      // Green

        exceeding: '#3b82f6',     // Blue

      },

    },
```


Page 205 of 349

```
      fontFamily: {
        sans: ['Inter', 'system-ui', 'sans-serif'],
      },
    },
  },
  plugins: [],
};
```

Testing

Unit Tests (Vitest)

```
// tests/unit/components/Button.test.tsx
import { describe, it, expect, vi } from 'vitest';
import { render, screen, fireEvent } from '@testing-library/react';
import { Button } from '@/components/ui/Button';
```


Page 206 of 349

```
describe('Button', () => {

  it('renders with correct text', () => {

    render(<Button />);

    expect(screen.getByText('Click me')).toBeInTheDocument();

  });




it('calls onClick when clicked', () => {

    const handleClick = vi.fn();

    render(<Button />);

    fireEvent.click(screen.getByText('Click me'));

    expect(handleClick).toHaveBeenCalledOnce();

  });




it('is disabled when isLoading is true', () => {

    render(<Button />);

    expect(screen.getByRole('button')).toBeDisabled();

  });

});
```


Page 207 of 349

E2E Tests (Playwright)

```
// tests/e2e/assessment.spec.ts

import { test, expect } from '@playwright/test';



test('teacher can create and submit assessment', async ({ page }) => {

  // Login

  await page.goto('/login');

  await page.fill('[name="email"]', 'teacher@school.edu');

  await page.fill('[name="password"]', 'password123');

  await page.click('button[type="submit"]');



  // Navigate to new assessment

  await page.click('text=New Assessment');



  // Fill form

  await page.selectOption('select[name="student"]', 'student-123');

  await page.selectOption('select[name="mode"]', 'explaining');

  await page.fill('textarea[name="prompt"]', 'Explain photosynthesis');



  // Submit

  await page.click('button[type="submit"]');
```


Page 208 of 349

```
  // Verify redirect to recording page
  await expect(page).toHaveURL(/\/assessments\/.*\/record/);
});
```

Database Schema

Entity Relationship Diagram

```
┌──────────────────┐     ┌──────────────────┐     ┌──────────────────┐
│     School       │     │      User        │     │     Student      │
├──────────────────┤     ├──────────────────┤     ├──────────────────┤
│ id (PK)          │◀────┤ school_id (FK)   │     │ id (PK)          │
│ name             │     │ id (PK)          │     │ user_id (FK)     │──┐
│ identifier       │     │ email            │     │ student_id       │  │
│ region           │     │ passwordhash     │     │ dateof_birth     │  │
│ createdat        │     │ role             │     │ yeargroup        │  │
└──────────────────┘     │ firstname        │     │ ageband          │  │
                         │ last_name        │     │ eal              │  │
                         │ isactive         │     │ firstlanguage    │  │
```


Page 209 of 349

```
                    └──────────────────┘      │ accommodations  │  │

                          │                   │ created_at      │  │

                          │                   └─────────────────┘  │

                          │                         │              │

                          │                         │              │

                          ▼                         ▼              │

                    ┌──────────────────┐      ┌─────────────────┐  │

                    │     Cohort       │      │   Enrollment    │  │

                    ├──────────────────┤      ├─────────────────┤  │

                    │ id (PK)          │◀─────┤ cohort_id (FK)  │  │

                    │ teacherid (FK)   │      │ studentid (FK)  │──┘

                    │ name             │      │ enrolled_at     │

                    │ year_group       │      └─────────────────┘

                    │ academic_year    │

                    └──────────────────┘

                          │

                          │

                          ▼

┌──────────────────────────────────────────────────────────────────────┐

│                            Assessment                                 │

├──────────────────────────────────────────────────────────────────────┤

│ id (PK, UUID)                                                         │

│ student_id (FK) ◀──────────────────────────────────────────────┐   │
```


Page 210 of 349

```
│ cohort_id (FK)                                                  │  │
│ mode (presenting|explaining|persuading)                         │  │
│ prompt (TEXT)                                                   │  │
│ timelimitseconds                                               │  │
│ status                                                          │  │
│ status_message                                                  │  │
│ consent_obtained                                                │  │
│ consent_date                                                    │  │
│ created_at                                                      │  │
│ uploaded_at                                                     │  │
│ completed_at                                                    │  │
└─────────────────────────────────────────────────────────────────────┘
                              │
        ┌─────────────────────┼─────────────────────┐
        │                     │                     │
        ▼                     ▼                     ▼
┌───────────────┐     ┌───────────────┐     ┌───────────────┐
│   Recording   │     │  Transcript   │     │FeatureSignals │
├───────────────┤     ├───────────────┤     ├───────────────┤
│ id (PK)       │     │ id (PK)       │     │ id (PK)       │
│ assessmentid │     │ assessmentid │     │ assessment_id │
│ s3_bucket     │     │ segments(JSON)│     │ wpm           │
│ s3key        │     │ fulltext     │     │ pause_ratio   │
```


Page 211 of 349

```
│ filesize      │     │ language      │     │ fillerfreq    │
│ duration_sec  │     │ confidence    │     │ ...features   │
│ audios3key    │     │ provider      │     │ quality_flag  │
│ videoquality  │     │ modelversion  │     │ extracted_at  │
│ audioquality  │     │ createdat     │     └───────────────┘
└───────────────┘     └───────────────┘

                            │
                            ▼

┌──────────────────────────────────────────────────────────────────────┐
│                        EvidenceCandidate                             │
├──────────────────────────────────────────────────────────────────────┤
│ id (PK)                                                              │
│ assessment_id (FK)                                                   │
│ candidateid (clip0, clip_1, etc.)                                   │
│ start_time (seconds)                                                 │
│ end_time (seconds)                                                   │
│ type (transcriptboundary, reasondense, etc.)                        │
│ summary                                                              │
│ features (JSON)                                                      │
└──────────────────────────────────────────────────────────────────────┘

                            │
                            ▼

┌──────────────────────────────────────────────────────────────────────┐
```


Page 212 of 349

```
│                            DraftReport                                │
├──────────────────────────────────────────────────────────────────────┤
│ id (PK)                                                               │
│ assessment_id (FK)                                                    │
│ physicalscore (JSON: band, confidence, evidenceclips, justification)  │
│ linguistic_score (JSON)                                               │
│ cognitive_score (JSON)                                                │
│ socialemotionalscore (JSON)                                           │
│ feedback (JSON: strengths[], next_steps[], goals[])                   │
│ ai_model                                                              │
│ generated_at                                                          │
│ is_reviewed                                                           │
└──────────────────────────────────────────────────────────────────────┘
                                │
                                ▼
┌──────────────────────────────────────────────────────────────────────┐
│                           SignedReport                                │
├──────────────────────────────────────────────────────────────────────┤
│ id (PK)                                                               │
│ assessment_id (FK)                                                    │
│ draft_id (FK)                                                         │
│ physical_score (JSON)                                                 │
│ linguistic_score (JSON)                                               │
```


Page 213 of 349

```
│ cognitive_score (JSON)                                          │
│ socialemotionalscore (JSON)                                     │
│ feedback (JSON)                                                 │
│ teacher_notes                                                   │
│ signed_by (FK → User)                                           │
│ signed_at                                                       │
└─────────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌─────────────────────────────────────────────────────────────────┐
│                           AuditLog                              │
├─────────────────────────────────────────────────────────────────┤
│ id (PK)                                                         │
│ report_id (FK)                                                  │
│ user_id (FK)                                                    │
│ action (scorechanged, clipchanged, feedback_edited, signed)     │
│ field                                                           │
│ old_value (JSON)                                                │
│ new_value (JSON)                                                │
│ timestamp                                                       │
└─────────────────────────────────────────────────────────────────┘
```


Page 214 of 349

```
┌──────────────────────────────────────────────────────────────────────┐
│                        BenchmarkVersion                              │
├──────────────────────────────────────────────────────────────────────┤
│ version (PK)                                                         │
│ age_band                                                             │
│ mode                                                                 │
│ definition (JSON - full BDL)                                         │
│ is_active                                                            │
│ created_at                                                           │
└──────────────────────────────────────────────────────────────────────┘
```

Table Definitions

Core Tables

`users` (via Django AbstractUser)

Page 215 of 349

```
CREATE TABLE users (

    id SERIAL PRIMARY KEY,

    email VARCHAR(254) UNIQUE NOT NULL,

    password VARCHAR(128) NOT NULL,  -- Django's hashed password

    first_name VARCHAR(150),

    last_name VARCHAR(150),

    role VARCHAR(20) NOT NULL CHECK (role IN ('admin', 'teacher', 'student')),

    school_id INTEGER REFERENCES schools(id) ON DELETE SET NULL,

    is_active BOOLEAN DEFAULT TRUE,

    date_joined TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

    last_login TIMESTAMP WITH TIME ZONE

);



CREATE INDEX idxusersschool ON users(school_id);

CREATE INDEX idxusersrole ON users(role);

CREATE INDEX idxusersemail ON users(email);
```

`schools`

```
CREATE TABLE schools (
```


Page 216 of 349

```
    id SERIAL PRIMARY KEY,

    name VARCHAR(200) NOT NULL,

    identifier VARCHAR(50) UNIQUE NOT NULL,  -- School URN or custom ID

    region VARCHAR(50),  -- For data residency

    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()

);



CREATE INDEX idxschoolsidentifier ON schools(identifier);
```

`students`

```
CREATE TABLE students (

    id SERIAL PRIMARY KEY,

    user_id INTEGER UNIQUE REFERENCES users(id) ON DELETE CASCADE,

    student_id VARCHAR(50) UNIQUE NOT NULL,  -- School's student ID

    dateofbirth DATE,

    year_group VARCHAR(10) NOT NULL,  -- "7", "8", "11", etc.

    age_band VARCHAR(10) NOT NULL,  -- "11-12", "13-14", etc.

    eal BOOLEAN DEFAULT FALSE,  -- English as Additional Language

    first_language VARCHAR(50),

    accommodations JSONB DEFAULT '{}',  -- Extended time, etc.
```


Page 217 of 349

```
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);



CREATE INDEX idxstudentsyeargroup ON students(yeargroup);
CREATE INDEX idxstudentsageband ON students(ageband);
CREATE INDEX idxstudentseal ON students(eal);
```

`cohorts`

```
CREATE TABLE cohorts (
    id SERIAL PRIMARY KEY,
    teacher_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    name VARCHAR(100) NOT NULL,
    year_group VARCHAR(10) NOT NULL,
    academic_year VARCHAR(9) NOT NULL,  -- "2025-2026"
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);



CREATE INDEX idxcohortsteacher ON cohorts(teacher_id);
```


Page 218 of 349

```
CREATE INDEX idxcohortsyear ON cohorts(academic_year);
```

`enrollments`

```
CREATE TABLE enrollments (

    id SERIAL PRIMARY KEY,

    student_id INTEGER NOT NULL REFERENCES students(id) ON DELETE CASCADE,

    cohort_id INTEGER NOT NULL REFERENCES cohorts(id) ON DELETE CASCADE,

    enrolled_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

    UNIQUE(studentid, cohortid)

);



CREATE INDEX idxenrollmentscohort ON enrollments(cohort_id);

CREATE INDEX idxenrollmentsstudent ON enrollments(student_id);
```

Assessment Tables

`assessments`

Page 219 of 349

```
CREATE TABLE assessments (

    id UUID PRIMARY KEY DEFAULT genrandomuuid(),

    student_id INTEGER NOT NULL REFERENCES students(id) ON DELETE CASCADE,

    cohort_id INTEGER NOT NULL REFERENCES cohorts(id) ON DELETE CASCADE,

    mode VARCHAR(20) NOT NULL CHECK (mode IN ('presenting', 'explaining', 'persuading'

    prompt TEXT NOT NULL,

    timelimitseconds INTEGER DEFAULT 180,

    status VARCHAR(20) NOT NULL DEFAULT 'pending' CHECK (

        status IN ('pending', 'uploading', 'processing', 'draft_ready',

                    'underreview', 'signedoff', 'error')

    ),

    status_message TEXT,

    consent_obtained BOOLEAN DEFAULT FALSE,

    consent_date TIMESTAMP WITH TIME ZONE,

    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

    uploaded_at TIMESTAMP WITH TIME ZONE,

    completed_at TIMESTAMP WITH TIME ZONE

);



CREATE INDEX idxassessmentsstudent ON assessments(student_id);

CREATE INDEX idxassessmentscohort ON assessments(cohort_id);

CREATE INDEX idxassessmentsstatus ON assessments(status);
```


Page 220 of 349

```
CREATE INDEX idxassessmentscreated ON assessments(created_at DESC);

CREATE INDEX idxassessmentsmode ON assessments(mode);





-- Composite index for teacher dashboard queries

CREATE INDEX idxassessmentscohortstatus ON assessments(cohortid, status, created_at DE
```

`recordings`

```
CREATE TABLE recordings (

    id SERIAL PRIMARY KEY,

    assessment_id UUID UNIQUE REFERENCES assessments(id) ON DELETE CASCADE,

    original_filename VARCHAR(255),

    s3_bucket VARCHAR(100) NOT NULL,

    s3_key VARCHAR(500) NOT NULL,

    filesizebytes BIGINT NOT NULL,

    duration_seconds FLOAT,

    audio_extracted BOOLEAN DEFAULT FALSE,

    audios3key VARCHAR(500),

    videoqualityscore FLOAT CHECK (videoqualityscore BETWEEN 0 AND 1),

    audioqualityscore FLOAT CHECK (audioqualityscore BETWEEN 0 AND 1),
```


Page 221 of 349

```
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()

);



CREATE INDEX idxrecordingsassessment ON recordings(assessment_id);
```

Analysis Tables

`transcripts`

```
CREATE TABLE transcripts (

    id SERIAL PRIMARY KEY,

    assessment_id UUID UNIQUE REFERENCES assessments(id) ON DELETE CASCADE,

    segments JSONB NOT NULL,  -- [{start, end, text, words: [{word, start, end, confid

    full_text TEXT NOT NULL,

    language VARCHAR(10) DEFAULT 'en-GB',

    confidence FLOAT CHECK (confidence BETWEEN 0 AND 1),

    provider VARCHAR(50) NOT NULL,  -- 'openai', 'aws'

    model_version VARCHAR(50),

    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()

);
```


Page 222 of 349

```
CREATE INDEX idxtranscriptsassessment ON transcripts(assessment_id);




-- Full-text search index

CREATE INDEX idxtranscriptsfts ON transcripts USING GIN (totsvector('english', fulltex
```

`feature_signals`

```
CREATE TABLE feature_signals (

    id SERIAL PRIMARY KEY,

    assessment_id UUID UNIQUE REFERENCES assessments(id) ON DELETE CASCADE,



    -- Physical features

    wpm FLOAT,

    pause_ratio FLOAT,

    filler_frequency FLOAT,

    volume_variance FLOAT,

    rhythm_stability FLOAT,



    -- Linguistic features
```


Page 223 of 349

```
    sentencelengthvariance FLOAT,
    vocabulary_diversity FLOAT,
    prompt_relevance FLOAT,
    clarity_score FLOAT,
    register_formality FLOAT,

    -- Cognitive features
    reason_density FLOAT,
    counterpoint_density FLOAT,
    logicalconnectordensity FLOAT,
    structure_completeness FLOAT,

    -- Social-emotional features
    audiencereferencefrequency FLOAT,
    intention_clarity FLOAT,
    sentiment_compound FLOAT,

    -- Metadata
    qualityflag VARCHAR(20) CHECK (qualityflag IN ('good', 'fair', 'poor')),
    extracted_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```


Page 224 of 349

```
CREATE INDEX idxfeaturesassessment ON featuresignals(assessmentid);
```

`evidence_candidates`

```
CREATE TABLE evidence_candidates (

    id SERIAL PRIMARY KEY,

    assessment_id UUID REFERENCES assessments(id) ON DELETE CASCADE,

    candidateid VARCHAR(20) NOT NULL,  -- "clip0", "clip_1", etc.

    start_time FLOAT NOT NULL,

    end_time FLOAT NOT NULL,

    type VARCHAR(50) NOT NULL,

    summary TEXT,

    features JSONB DEFAULT '{}',

    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),



    UNIQUE(assessmentid, candidateid)

);



CREATE INDEX idxcandidatesassessment ON evidencecandidates(assessmentid);

CREATE INDEX idxcandidatestype ON evidence_candidates(type);
```


Page 225 of 349

Report Tables

`draft_reports`

```
CREATE TABLE draft_reports (

    id SERIAL PRIMARY KEY,

    assessment_id UUID UNIQUE REFERENCES assessments(id) ON DELETE CASCADE,



    -- Strand scores (JSON structure)

    physicalscore JSONB NOT NULL,  -- {band, confidence, evidenceclips: [], justificat

    linguistic_score JSONB NOT NULL,

    cognitive_score JSONB NOT NULL,

    socialemotionalscore JSONB NOT NULL,



    -- Feedback

    feedback JSONB NOT NULL,  -- {strengths: [], next_steps: [], goals: []}



    -- Metadata

    ai_model VARCHAR(50) NOT NULL,

    generated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

    is_reviewed BOOLEAN DEFAULT FALSE
```


Page 226 of 349

```
);



CREATE INDEX idxdraftreportsassessment ON draftreports(assessment_id);

CREATE INDEX idxdraftreportsgenerated ON draftreports(generated_at DESC);
```

`signed_reports`

```
CREATE TABLE signed_reports (

    id SERIAL PRIMARY KEY,

    assessment_id UUID UNIQUE REFERENCES assessments(id) ON DELETE CASCADE,

    draftid INTEGER REFERENCES draftreports(id) ON DELETE SET NULL,



    -- Final scores (may differ from draft)

    physical_score JSONB NOT NULL,

    linguistic_score JSONB NOT NULL,

    cognitive_score JSONB NOT NULL,

    socialemotionalscore JSONB NOT NULL,



    -- Final feedback

    feedback JSONB NOT NULL,
```


Page 227 of 349

```
    teacher_notes TEXT,



    -- Signing

    signed_by INTEGER REFERENCES users(id) ON DELETE SET NULL,

    signed_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()

);



CREATE INDEX idxsignedreportsassessment ON signedreports(assessment_id);

CREATE INDEX idxsignedreportssignedby ON signedreports(signedby);

CREATE INDEX idxsignedreportssignedat ON signedreports(signedat DESC);
```

`audit_logs`

```
CREATE TABLE audit_logs (

    id BIGSERIAL PRIMARY KEY,

    reportid INTEGER REFERENCES draftreports(id) ON DELETE CASCADE,

    user_id INTEGER REFERENCES users(id) ON DELETE SET NULL,

    action VARCHAR(20) NOT NULL CHECK (

        action IN ('scorechanged', 'clipchanged', 'feedback_edited', 'signed')

    ),

    field VARCHAR(50),
```


Page 228 of 349

```
    old_value JSONB,
    new_value JSONB,
    timestamp TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);



CREATE INDEX idxauditlogsreport ON auditlogs(report_id);
CREATE INDEX idxauditlogsuser ON auditlogs(user_id);
CREATE INDEX idxauditlogstimestamp ON auditlogs(timestamp DESC);





-- Partition by month for large scale (future optimization)
-- CREATE TABLE auditlogsy2026m03 PARTITION OF audit_logs...
```

Benchmark Tables

`benchmark_versions`

```
CREATE TABLE benchmark_versions (
    version VARCHAR(20) NOT NULL,
    age_band VARCHAR(10) NOT NULL,
```


Page 229 of 349

```
    mode VARCHAR(20) NOT NULL,

    definition JSONB NOT NULL,

    is_active BOOLEAN DEFAULT TRUE,

    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),



    PRIMARY KEY (version, age_band, mode)

);



CREATE INDEX idxbenchmarksactive ON benchmarkversions(isactive);

CREATE INDEX idxbenchmarksagemode ON benchmarkversions(age_band, mode);
```

Indexes Summary

Performance-Critical Indexes

Table	Index	Purpose

Page 230 of 349

assessments	`idx_assessments_cohort_status`	Teacher dashboard queries
assessments	`idx_assessments_student`	Student history queries
assessments	`idx_assessments_status`	Processing queue queries
draft_reports	`idx_draft_reports_generated`	Recent reports listing
audit_logs	`idx_audit_logs_timestamp`	Audit trail queries
transcripts	`idx_transcripts_fts`	Full-text search on transcripts

Foreign Key Indexes (Auto-created, but explicit for documentation)

Table	Column	References
users	school_id	schools(id)
students	user_id	users(id)
cohorts	teacher_id	users(id)
enrollments	student_id	students(id)
enrollments	cohort_id	cohorts(id)
assessments	student_id	students(id)
assessments	cohort_id	cohorts(id)
recordings	assessment_id	assessments(id)
transcripts	assessment_id	assessments(id)
features_signals	assessment_id	assessments(id)
evidence_candidates	assessment_id	assessments(id)
draft_reports	assessment_id	assessments(id)
signed_reports	assessment_id	assessments(id)
signed_reports	draft_id	draft_reports(id)
signed_reports	signed_by	users(id)
audit_logs	report_id	draft_reports(id)
audit_logs	user_id	users(id)

Page 231 of 349

Constraints & Validation

Check Constraints

```
-- Band values must be valid

ALTER TABLE draft_reports

ADD CONSTRAINT chkphysicalband

CHECK (physicalscore->>'band' IN ('emerging', 'expected', 'exceeding', 'insufficientev




-- Confidence values between 0 and 1

ALTER TABLE draft_reports

ADD CONSTRAINT chkphysicalconfidence

CHECK ((physical_score->>'confidence')::float BETWEEN 0 AND 1);




-- Time values must be positive

ALTER TABLE evidence_candidates

ADD CONSTRAINT chkpositivetimes

CHECK (starttime >= 0 AND endtime > start_time);
```


Page 232 of 349

```
-- File sizes must be positive

ALTER TABLE recordings

ADD CONSTRAINT chkpositivefile_size

CHECK (filesizebytes > 0);
```

Data Retention & Archival

Retention Policies

Data Type	Retention Period	Action
Video recordings	7 years	S3 Glacier after 90 days
Transcripts	7 years	Keep in DB
Feature signals	1 year	Archive to S3 after 1 year
Draft reports	7 years	Soft delete only
Signed reports	Permanent	Never delete
Audit logs	7 years	Partition and archive

Page 233 of 349

Soft Delete Implementation

```
-- Add soft delete columns to key tables

ALTER TABLE assessments ADD COLUMN deleted_at TIMESTAMP WITH TIME ZONE;

ALTER TABLE students ADD COLUMN deleted_at TIMESTAMP WITH TIME ZONE;




-- Create view for active records

CREATE VIEW active_assessments AS

SELECT * FROM assessments WHERE deleted_at IS NULL;




-- Create partial index for active records

CREATE INDEX idx*assessments*active ON assessments(id) WHERE deleted_at IS NULL;
```

Migration Strategy

Page 234 of 349

Initial Migration Order

1. `0001_initial.py` - Schools, Users (auth tables)
2. `0002_students.py` - Students
3. `0003_cohorts.py` - Cohorts and Enrollments
4. `0004_assessments.py` - Assessments and Recordings
5. `0005_analysis.py` - Transcripts, Features, Evidence
6. `0006_reports.py` - Draft, Signed Reports
7. `0007_audit.py` - Audit Logs
8. `0008_benchmarks.py` - Benchmark Versions

Data Migration for BDL

```
# migrations/0008_load_benchmarks.py

from django.db import migrations




def load_benchmarks(apps, schema_editor):

    BenchmarkVersion = apps.get_model('benchmarks', 'BenchmarkVersion')



    # Load from JSON files

    import json

    from pathlib import Path



    bdl_dir = Path(__file__).parent / 'bdl'
```


Page 235 of 349

```
    for filepath in bdldir.glob('*/.json'):
        with open(file_path) as f:
            definition = json.load(f)


        BenchmarkVersion.objects.getorcreate(
            version=definition['version'],
            ageband=definition['ageband'],
            mode=definition['mode'],
            defaults={'definition': definition}
        )




class Migration(migrations.Migration):
    dependencies = [
        ('benchmarks', '0007_benchmarkversion'),
    ]


    operations = [
        migrations.RunPython(load_benchmarks),
    ]
```

Security & Compliance

Page 236 of 349

Overview

Oracy AI handles sensitive educational data including video recordings of minors. This document outlines the security architecture and compliance measures.

Authentication & Authorization

JWT-Based Authentication

```
# config/settings/base.py

REST_FRAMEWORK = {

    'DEFAULTAUTHENTICATIONCLASSES': [

        'restframeworksimplejwt.authentication.JWTAuthentication',

    ],

    'DEFAULTPERMISSIONCLASSES': [

        'rest_framework.permissions.IsAuthenticated',
```


Page 237 of 349

```
    ],
}


SIMPLE_JWT = {
    'ACCESSTOKENLIFETIME': timedelta(minutes=15),
    'REFRESHTOKENLIFETIME': timedelta(days=7),
    'ROTATEREFRESHTOKENS': True,
    'BLACKLISTAFTERROTATION': True,
    'ALGORITHM': 'HS256',
    'SIGNINGKEY': SECRETKEY,
    'AUTHHEADERTYPES': ('Bearer',),
}
```

Password Security

```
# Use Argon2 for password hashing
PASSWORD_HASHERS = [
    'django.contrib.auth.hashers.Argon2PasswordHasher',
    'django.contrib.auth.hashers.PBKDF2PasswordHasher',
]
```


Page 238 of 349

Password validation

```
AUTHPASSWORDVALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
        'OPTIONS': {'min_length': 12}
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'apps.users.validators.CustomPasswordValidator',
    },
]
```

Role-Based Access Control (RBAC)

```
# apps/users/permissions.py
from rest_framework import permissions
```


Page 239 of 349

```
class IsAdmin(permissions.BasePermission):
    def has_permission(self, request, view):
        return request.user.role == 'admin'




class IsTeacher(permissions.BasePermission):
    def has_permission(self, request, view):
        return request.user.role == 'teacher'




class IsOwnerOrAdmin(permissions.BasePermission):
    """Allow if user is admin or owns the resource."""
    def has_object_permission(self, request, view, obj):
        if request.user.role == 'admin':
            return True
        return obj.user == request.user




class CanAccessStudent(permissions.BasePermission):
    """Teachers can only access students in their cohorts."""
    def has_object_permission(self, request, view, obj):
```


Page 240 of 349

```
        if request.user.role == 'admin':
            return True
        # Check if student is in any of teacher's cohorts
        return obj.enrollments.filter(
            cohort__teacher=request.user
        ).exists()
```

Object-Level Permissions

```
# apps/assessments/views.py
from django.shortcuts import getobjector_404


class AssessmentViewSet(viewsets.ModelViewSet):
    serializer_class = AssessmentSerializer

    def get_queryset(self):
        """Teachers only see assessments from their cohorts."""
        user = self.request.user

        if user.role == 'admin':
```


Page 241 of 349

```
            return Assessment.objects.all()


        if user.role == 'teacher':
            return Assessment.objects.filter(
                cohort__teacher=user
            )


        # Students only see their own
        return Assessment.objects.filter(
            student__user=user
        )


    def perform_create(self, serializer):
        """Ensure teacher owns the cohort they're creating for."""
        cohort = serializer.validated_data['cohort']


        if self.request.user.role == 'teacher' and cohort.teacher != self.request.user
            raise PermissionDenied("You don't have access to this cohort.")


        serializer.save()
```


Page 242 of 349

Video Storage Security

S3 Security Configuration

```
# config/settings/production.py


# Private bucket - no public access


AWSS3SIGNATURE_VERSION = 's3v4'

AWSDEFAULTACL = 'private'

AWSS3FILE_OVERWRITE = False
```


Page 243 of 349

Server-side encryption

```
AWSS3ENCRYPTION = True

AWSS3KMSKEYID = os.getenv('AWSKMSKEY_ID')  # Customer-managed key
```


Page 244 of 349

```
# Presigned URL expiration (1 hour for viewin


AWSQUERYSTRINGEXPIRE = 3600

AWSQUERYSTRINGAUTH = True
```

Bucket Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyNonSSL",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:*",
      "Resource": [
        "arn:aws:s3:::oracy-prod-media/*",
        "arn:aws:s3:::oracy-prod-media"
      ],
      "Condition": {
```


Page 245 of 349

```
        "Bool": {
          "aws:SecureTransport": "false"
        }
      }
    },
    {
      "Sid": "DenyIncorrectKMSKey",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::oracy-prod-media/*",
      "Condition": {
        "StringNotEquals": {
          "s3:x-amz-server-side-encryption-aws-kms-key-id": "arn:aws:kms:REGION:ACCOUN
        }
      }
    },
    {
      "Sid": "AllowECSAccess",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::ACCOUNT:role/oracy-ecs-task-role"
      },
```


Page 246 of 349

```
      "Action": [
        "s3:PutObject",
        "s3:GetObject",
        "s3:DeleteObject"
      ],
      "Resource": "arn:aws:s3:::oracy-prod-media/*",
      "Condition": {
        "StringEquals": {
          "s3:x-amz-server-side-encryption": "aws:kms"
        }
      }
    }
  ]
}
```

Presigned URL Implementation

```
# apps/assessments/services/storage.py
import boto3
from botocore.exceptions import ClientError
from django.conf import settings
```


Page 247 of 349

```
class SecureVideoStorage:
    def init(self):
        self.s3 = boto3.client('s3')
        self.bucket = settings.AWSSTORAGEBUCKET_NAME

    def generateuploadurl(
        self,
        assessment_id: str,
        filename: str,
        content_type: str,
        file_size: int,
        expiration: int = 900  # 15 minutes
    ) -> dict:
        """Generate presigned URL for direct browser upload."""

        # Sanitize filename
        safefilename = self.sanitize_filename(filename)
        s3key = f"uploads/{assessmentid}/{safe_filename}"

        # Size limit validation (500MB)
        if file_size > 500  1024  1024:
```


Page 248 of 349

```
            raise ValueError("File too large (max 500MB)")



        try:

            url = self.s3.generatepresignedurl(

                'put_object',

                Params={

                    'Bucket': self.bucket,

                    'Key': s3_key,

                    'ContentType': content_type,

                    'ServerSideEncryption': 'aws:kms',

                    'SSEKMSKeyId': settings.AWSS3KMSKEYID,

                },

                ExpiresIn=expiration

            )



            return {

                'uploadUrl': url,

                's3Key': s3_key,

                'expiresIn': expiration

            }

        except ClientError as e:

            logger.error(f"Failed to generate upload URL: {e}")

            raise
```


Page 249 of 349

```
def generateviewurl(
    self,
    s3_key: str,
    expiration: int = 3600  # 1 hour
) -> str:
    """Generate time-limited viewing URL."""

    try:
        url = self.s3.generatepresignedurl(
            'get_object',
            Params={
                'Bucket': self.bucket,
                'Key': s3_key,
            },
            ExpiresIn=expiration
        )
        return url
    except ClientError as e:
        logger.error(f"Failed to generate view URL: {e}")
        raise

def movetopermanent(self, temps3key: str, assessment_id: str) -> str:
```


Page 250 of 349

```
        """Move from uploads/ to permanent recordings/ location."""



        # Parse date for organization

        from datetime import datetime

        now = datetime.utcnow()



        permanent_key = (

            f"recordings/{now.year}/{now.month:02d}/"

            f"{assessment_id}/video.mp4"

        )



        # Copy object

        self.s3.copy_object(

            CopySource={'Bucket': self.bucket, 'Key': temps3key},

            Bucket=self.bucket,

            Key=permanent_key,

            ServerSideEncryption='aws:kms',

            SSEKMSKeyId=settings.AWSS3KMSKEYID,

        )



        # Delete temporary

        self.s3.deleteobject(Bucket=self.bucket, Key=temps3_key)
```


Page 251 of 349

```
        return permanent_key
```

Access Logging

```
# Enable S3 access logging

AWSS3LOGGING = {

    'bucket': 'oracy-access-logs',

    'prefix': 's3-access/'

}


# Log all presigned URL generations


class VideoAccessLogger:

    def logaccess(self, userid: str, assessment_id: str, action: str):

        AuditLog.objects.create(

            userid=userid,

            assessmentid=assessmentid,

            action=f'video_{action}',

            timestamp=timezone.now(),

            ipaddress=self.request.META.get('REMOTEADDR'),
```


Page 252 of 349

```
            useragent=self.request.META.get('HTTPUSER_AGENT'),

        )
```

Data Protection (GDPR)

Data Classification

Data Type	Classification	Encryption at Rest	Encryption in Transit
Video recordings	High (Special Category)	KMS (AES-256)	TLS 1.3
Student PII	High	KMS (AES-256)	TLS 1.3
Transcripts	Medium	KMS (AES-256)	TLS 1.3
Assessment scores	Medium	RDS encryption	TLS 1.3
Audit logs	High	S3-SSE	TLS 1.3

Consent Management

```
# apps/assessments/models.py
```


Page 253 of 349

```
class Assessment(models.Model):
    # ... fields ...

    consent_obtained = models.BooleanField(default=False)
    consent_date = models.DateTimeField(null=True, blank=True)
    consentrecordedby = models.ForeignKey(
        User,
        ondelete=models.SETNULL,
        null=True,
        relatedname='recordedconsents'
    )

    # Consent can be withdrawn
    consent_withdrawn = models.BooleanField(default=False)
    consentwithdrawnat = models.DateTimeField(null=True, blank=True)
    consentwithdrawnby = models.ForeignKey(
        User,
        ondelete=models.SETNULL,
        null=True,
        relatedname='withdrawnconsents'
    )
```


Page 254 of 349

```
    def withdrawconsent(self, withdrawnby: User):
        """Handle consent withdrawal - triggers data deletion."""
        self.consent_withdrawn = True
        self.consentwithdrawnat = timezone.now()
        self.consentwithdrawnby = withdrawn_by
        self.save()


        # Trigger deletion workflow
        from .tasks import deleteassessmentdata
        deleteassessmentdata.delay(self.id)
```

Right to Erasure (GDPR Article 17)

```
# apps/assessments/tasks.py



@app.task
def deleteassessmentdata(assessment_id: str):
    """Complete deletion of assessment data (GDPR erasure)."""


    assessment = Assessment.objects.get(id=assessment_id)
```


Page 255 of 349

```
# 1. Delete video from S3
if assessment.recording:
    storage = SecureVideoStorage()
    storage.s3.delete_object(
        Bucket=settings.AWSSTORAGEBUCKET_NAME,
        Key=assessment.recording.s3_key
    )
    if assessment.recording.audios3key:
        storage.s3.delete_object(
            Bucket=settings.AWSSTORAGEBUCKET_NAME,
            Key=assessment.recording.audios3key
        )

# 2. Delete evidence clips
for clip in assessment.evidence_clips.all():
    storage.s3.delete_object(
        Bucket=settings.AWSSTORAGEBUCKET_NAME,
        Key=clip.s3_key
    )

# 3. Soft delete database records (retain audit log)
assessment.deleted_at = timezone.now()
```


Page 256 of 349

```
    assessment.save()



    # 4. Anonymize student reference in reports

    SignedReport.objects.filter(assessment=assessment).update(

        anonymized=True,

        studentidentifierhash=hashlib.sha256(

            str(assessment.student_id).encode()

        ).hexdigest()[:16]

    )
```

Data Export (GDPR Article 20 - Portability)

```
# apps/reports/export.py



class GDPRDataExporter:

    """Export all data for a student (GDPR portability)."""



    def exportstudentdata(self, student_id: int) -> dict:

        student = Student.objects.get(id=student_id)
```


Page 257 of 349

```
        return {
            'student_profile': {
                'studentid': student.studentid,
                'yeargroup': student.yeargroup,
                'ageband': student.ageband,
                'eal': student.eal,
                'firstlanguage': student.firstlanguage,
            },
            'assessments': [
                {
                    'id': str(a.id),
                    'mode': a.mode,
                    'prompt': a.prompt,
                    'createdat': a.createdat.isoformat(),
                    'transcript': a.transcript.full_text if hasattr(a, 'transcript') e
                    'scores': {
                        'physical': a.signedreport.physicalscore if hasattr(a, 'signed
                        'linguistic': a.signedreport.linguisticscore if hasattr(a, 'si
                        'cognitive': a.signedreport.cognitivescore if hasattr(a, 'sign
                        'socialemotional': a.signedreport.socialemotionalscore if hasa
                    },
                    'feedback': a.signedreport.feedback if hasattr(a, 'signedreport')
                }
```


Page 258 of 349

```
                for a in Assessment.objects.filter(student=student)

            ],

            'exportgeneratedat': timezone.now().isoformat(),

        }
```

Audit Logging

Comprehensive Audit Trail

```
# apps/audit/models.py



class AuditLog(models.Model):

    """Comprehensive audit log for all data changes."""



    ACTION_CHOICES = [

        ('create', 'Create'),
```


Page 259 of 349

```
        ('update', 'Update'),
        ('delete', 'Delete'),
        ('view', 'View'),
        ('export', 'Export'),
        ('sign', 'Sign Report'),
        ('login', 'Login'),
        ('logout', 'Logout'),
        ('video_access', 'Video Access'),
        ('consent_withdraw', 'Consent Withdrawn'),
    ]

    id = models.BigAutoField(primary_key=True)
    timestamp = models.DateTimeField(default=timezone.now, db_index=True)

    # Who
    user = models.ForeignKey(
        User,
        ondelete=models.SETNULL,
        null=True,
        relatedname='auditlogs'
    )
    user_email = models.EmailField()  # Snapshot in case user is deleted
```


Page 260 of 349

```
    # What
    action = models.CharField(maxlength=20, choices=ACTIONCHOICES)
    resourcetype = models.CharField(maxlength=50)  # 'Assessment', 'Report', etc.
    resourceid = models.CharField(maxlength=100)

    # Details
    field = models.CharField(max_length=100, blank=True)
    old_value = models.JSONField(null=True)
    new_value = models.JSONField(null=True)

    # Context
    ip_address = models.GenericIPAddressField(null=True)
    user_agent = models.TextField(blank=True)
    sessionid = models.CharField(maxlength=100, blank=True)

    class Meta:
        indexes = [
            models.Index(fields=['timestamp']),
            models.Index(fields=['user', 'timestamp']),
            models.Index(fields=['resourcetype', 'resourceid']),
            models.Index(fields=['action', 'timestamp']),
        ]
        ordering = ['-timestamp']
```


Page 261 of 349

Middleware for automatic logging

```
class AuditMiddleware:

    def init(self, get_response):

        self.getresponse = getresponse



    def call(self, request):

        response = self.get_response(request)



        # Log specific actions

        if request.user.is_authenticated:

            self.logrequest(request, response)



        return response



    def logrequest(self, request, response):

        # Log video access

        if 'recordings' in request.path and request.method == 'GET':

            AuditLog.objects.create(

                user=request.user,
```


Page 262 of 349

```
                user_email=request.user.email,
                action='video_access',
                resource_type='Recording',
                resourceid=request.resolvermatch.kwargs.get('id', 'unknown'),
                ipaddress=self.getclientip(request),
                useragent=request.META.get('HTTPUSER_AGENT', '')[:500],
            )


    def getclient_ip(self, request):
        xforwardedfor = request.META.get('HTTPXFORWARDED_FOR')
        if xforwardedfor:
            return xforwardedfor.split(',')[0].strip()
        return request.META.get('REMOTE_ADDR')
```

Model Change Tracking

```
# apps/core/models.py


from djangocurrentuser.middleware import getcurrent_user
```


Page 263 of 349

```
class AuditedModel(models.Model):
    """Abstract base model with automatic audit logging."""

    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
    created_by = models.ForeignKey(
        User,
        on_delete=models.SET_NULL,
        null=True,
        related_name='%(class)s_created'
    )
    updated_by = models.ForeignKey(
        User,
        on_delete=models.SET_NULL,
        null=True,
        related_name='%(class)s_updated'
    )

    class Meta:
        abstract = True

    def save(self, *args, **kwargs):
```


Page 264 of 349

```
        user = getcurrentuser()

        if user and user.is_authenticated:

            if not self.pk:

                self.created_by = user

            self.updated_by = user

        super().save(args, *kwargs)
```

Network Security

TLS Configuration

```
# Production settings

SECURESSLREDIRECT = True

SECUREPROXYSSLHEADER = ('HTTPXFORWARDEDPROTO', 'https')

SESSIONCOOKIESECURE = True

CSRFCOOKIESECURE = True
```


Page 265 of 349

```
SECUREHSTSSECONDS = 31536000  # 1 year

SECUREHSTSINCLUDE_SUBDOMAINS = True

SECUREHSTSPRELOAD = True

SECURECONTENTTYPE_NOSNIFF = True

SECUREBROWSERXSS_FILTER = True

XFRAMEOPTIONS = 'DENY'
```

AWS WAF Configuration

```
# Terraform configuration for WAF

resource "awswafv2web_acl" "main" {

  name        = "oracy-waf"

  description = "WAF rules for Oracy AI"

  scope       = "REGIONAL"



default_action {

    allow {}

  }
```


Page 266 of 349

```
# Rate limiting

  rule {

    name     = "RateLimit"

    priority = 1




action {

      block {}

    }




statement {

      rate based statement {

        limit              = 2000

        aggregate key type = "IP"

      }

    }




visibility_config {

      cloudwatch metrics enabled = true

      metric_name                = "RateLimitRule"

      sampled requests enabled   = true
```


Page 267 of 349

```
    }
  }



# SQL injection protection
  rule {
    name     = "SQLInjection"
    priority = 2



override_action {
      none {}
    }



statement {
      managedrulegroup_statement {
        name        = "AWSManagedRulesSQLiRuleSet"
        vendor_name = "AWS"
      }
    }
```


Page 268 of 349

```
visibility_config {
      cloudwatchmetricsenabled = true
      metric_name                = "SQLInjectionRule"
      sampledrequestsenabled   = true
    }
  }


# Known bad actors
  rule {
    name     = "BadActors"
    priority = 3


override_action {
      none {}
    }


statement {
      managedrulegroup_statement {
```


Page 269 of 349

```
        name        = "AWSManagedRulesKnownBadInputsRuleSet"

        vendor_name = "AWS"

      }

    }





visibility_config {

      cloudwatchmetricsenabled = true

      metric_name                = "BadActorsRule"

      sampledrequestsenabled   = true

    }

  }

}
```

Secrets Management

AWS Secrets Manager

Page 270 of 349

```
# config/settings/production.py

import boto3

import json



def getsecret(secretname):

    """Retrieve secret from AWS Secrets Manager."""

    client = boto3.client('secretsmanager')

    response = client.getsecretvalue(SecretId=secret_name)

    return json.loads(response['SecretString'])
```

Load secrets

```
secrets = get_secret('oracy/production')



SECRETKEY = secrets['djangosecret_key']

DATABASES = {

    'default': {

        'ENGINE': 'django.db.backends.postgresql',

        'NAME': secrets['db_name'],
```


Page 271 of 349

```
        'USER': secrets['db_user'],

        'PASSWORD': secrets['db_password'],

        'HOST': secrets['db_host'],

        'PORT': secrets['db_port'],

    }

}

OPENAIAPIKEY = secrets['openaiapikey']
```

Environment Variable Validation

```
# config/settings/base.py

import environ



env = environ.Env(

    DEBUG=(bool, False),

    SECRET_KEY=(str, None),

    ALLOWED_HOSTS=(list, []),

)
```


Page 272 of 349

Required environment variables

```
REQUIRED_ENV = [
    'SECRET_KEY',
    'DATABASE_URL',
    'REDIS_URL',
    'AWSSTORAGEBUCKET_NAME',
]


for var in REQUIRED_ENV:
    if not env(var):
        raise ImproperlyConfigured(f'{var} environment variable is required')
```

Compliance Checklist

Page 273 of 349

GDPR Compliance

- [ ] **Lawful basis** - Consent obtained before recording
- [ ] **Right to be informed** - Privacy notice provided
- [ ] **Right of access** - Data export functionality
- [ ] **Right to rectification** - Edit capabilities for teachers
- [ ] **Right to erasure** - Complete deletion workflow
- [ ] **Right to restrict processing** - Pause processing capability
- [ ] **Right to data portability** - JSON/CSV export
- [ ] **Right to object** - Objection handling process
- [ ] **Data minimization** - Only collect necessary data
- [ ] **Purpose limitation** - Clear purpose defined
- [ ] **Storage limitation** - Retention policies enforced
- [ ] **Security** - Encryption and access controls
- [ ] **Accountability** - Audit logging
- [ ] **DPO** - Data Protection Officer appointed

Page 274 of 349

Children's Data (UK GDPR)

- [ ] Age verification for consent
- [ ] Parental consent for under-13s
- [ ] Clear privacy notices for children
- [ ] Enhanced security measures
- [ ] Regular data protection impact assessments (DPIA)

Data Residency

- [ ] Primary region: EU (for EU schools)
- [ ] Backup region: Same jurisdiction
- [ ] Cross-border transfer safeguards

Page 275 of 349

Incident Response

Security Incident Procedure

1. **Detection** - CloudWatch alarms, WAF logs, audit logs
2. **Assessment** - Severity classification
3. **Containment** - Isolate affected systems
4. **Investigation** - Root cause analysis
5. **Remediation** - Fix vulnerability
6. **Recovery** - Restore services
7. **Post-incident** - Lessons learned

Data Breach Notification

- **Within 72 hours** - Notify ICO (if applicable)
- **Without undue delay** - Notify affected individuals
- **Documentation** - All breaches recorded

Contact

Page 276 of 349

- **Security team**: security@oracy.ai
- **DPO**: dpo@oracy.ai
- **Incident hotline**: +44 XXX XXX XXXX

Development Roadmap

MVP Timeline (2 Weeks)

Week 1: Foundation & AI Pipeline

Day 1-2: Project Setup & Infrastructure

Deliverables:

- [ ] Initialize Django project with apps structure
- [ ] Initialize React project with Vite + TypeScript
- [ ] Docker Compose for local development
- [ ] AWS dev environment setup (VPC, ECS cluster, RDS)

Page 277 of 349

- [ ] CI/CD pipeline skeleton (GitHub Actions)

Key Decisions:

- Django project structure finalized
- Database schema v1 implemented
- AWS region selected (ap-southeast-1 for initial dev)

Day 3-4: Authentication & Core Models

Deliverables:

- [ ] User authentication (JWT) - backend
- [ ] User login/logout - frontend
- [ ] User model with roles (admin, teacher)
- [ ] School model
- [ ] Database migrations

Page 278 of 349

APIs:

- `POST /api/auth/login/`
- `POST /api/auth/refresh/`
- `GET /api/users/me/`

Frontend:

- Login page
- Auth context/store
- Protected routes

Day 5-6: Student Management & Assessment Creation

Deliverables:

- [ ] Student model and CRUD APIs
- [ ] Cohort model and enrollment

Page 279 of 349

- [ ] Assessment creation flow
- [ ] Prompt templates

APIs:

- `GET/POST /api/students/`
- `GET/POST /api/cohorts/`
- `GET/POST /api/assessments/`
- `GET /api/prompts/`

Frontend:

- Students list page
- Student detail page
- Assessment creation form
- Cohort selector

Page 280 of 349

Day 7: Video Upload & Storage

Deliverables:

- [ ] S3 bucket configuration with encryption
- [ ] Presigned URL generation
- [ ] Video upload with progress (frontend)
- [ ] Recording model and storage

APIs:

- `POST /api/assessments/{id}/upload-url/`
- `POST /api/assessments/{id}/complete-upload/`

Frontend:

- Video uploader component
- Progress indicator
- Upload completion handling

Page 281 of 349

Week 2: AI Pipeline & Review UI

Day 8-9: Speech-to-Text & Feature Extraction

Deliverables:

- [ ] STT service integration (OpenAI Whisper)
- [ ] Transcript model and storage
- [ ] Feature extraction pipeline (Physical, Linguistic, Cognitive, Social)
- [ ] FeatureSignals model

Celery Tasks:

- `extract*audio*task`
- `transcribe_task`
- `extract*features*task`

Page 282 of 349

Storage:

- Transcript JSON in PostgreSQL
- Extracted features in FeatureSignals table

Day 10: Benchmarking & Evidence Generation

Deliverables:

- [ ] BDL v0.1 JSON definitions (3 modes × 3-4 age bands)
- [ ] Benchmark engine (BDL interpreter)
- [ ] Evidence candidate generation
- [ ] EvidenceCandidate model

Files:

- `benchmarks/bdl/v1.0.0/11-12_explaining.json`
- `benchmarks/bdl/v1.0.0/11-12_presenting.json`

Page 283 of 349

- `benchmarks/bdl/v1.0.0/11-12_persuading.json`
- (Add more age bands as needed)

Day 11: LLM Scoring Service

Deliverables:

- [ ] LLM scoring integration (OpenAI GPT-4o)
- [ ] Structured JSON output parsing
- [ ] DraftReport generation
- [ ] Evidence clip selection validation

Prompt Engineering:

- Rubric-constrained scoring prompt
- Mode-specific feedback generation

Page 284 of 349

Celery Task:

- `scorewithllm_task`

Day 12: Teacher Review UI

Deliverables:

- [ ] Draft report display
- [ ] Video player with clip navigation
- [ ] Score editing interface
- [ ] Evidence clip replacement/addition
- [ ] Feedback editing

Frontend Components:

- `ReportViewer`
- `ScoreEditor`
- `EvidenceClips`

Page 285 of 349

- `FeedbackSection`

APIs:

- `GET /api/reports/draft/{id}/`
- `POST /api/reports/draft/{id}/edit/`

Day 13: Sign-off, Audit & PDF Export

Deliverables:

- [ ] SignedReport model and workflow
- [ ] Audit logging (all changes tracked)
- [ ] PDF report generation
- [ ] Sign-off confirmation

APIs:

Page 286 of 349

- `POST /api/reports/{id}/sign/`
- `GET /api/reports/{id}/export/pdf/`

Frontend:

- Sign-off modal
- PDF download
- Audit trail view

Day 14: Dashboard, Polish & Integration

Deliverables:

- [ ] Teacher dashboard
- [ ] Assessment status tracking
- [ ] WebSocket real-time updates
- [ ] Error handling and retry logic
- [ ] End-to-end testing

Page 287 of 349

Dashboard Features:

- Recent assessments list
- Status indicators
- Processing queue
- Quick actions

Testing:

- Full workflow test
- Video upload → AI processing → Review → Sign-off

Post-MVP Phases

Phase 1: Stabilization (Weeks 3-4)

Page 288 of 349

Goals:

- Bug fixes and performance optimization
- Security hardening
- Testing coverage improvement
- Documentation completion

Tasks:

- [ ] Unit test coverage > 80%
- [ ] Integration tests for API
- [ ] E2E tests with Playwright
- [ ] Security audit
- [ ] Load testing
- [ ] Error monitoring (Sentry)
- [ ] Performance monitoring

Page 289 of 349

Phase 2: Feature Expansion (Weeks 5-8)

Student-Facing Features:

- [ ] Student dashboard (view own progress)
- [ ] Practice mode (self-assessment)
- [ ] Goal tracking

Teacher Features:

- [ ] Bulk assessment creation
- [ ] Cohort analytics
- [ ] Progress tracking over time
- [ ] Comparison views

Administrative:

- [ ] School admin dashboard
- [ ] User management

Page 290 of 349

- [ ] Benchmark version management UI

Technical:

- [ ] Caching layer optimization
- [ ] CDN for video streaming
- [ ] Background job monitoring

Phase 3: Custom Model Training (Months 3-6)

Prerequisites:

- Minimum 1000 teacher-reviewed assessments
- Sufficient variation in age bands and modes

Phase 3.1: Rubric Classifier (Week 9-12)

- [ ] Feature engineering from teacher edits

Page 291 of 349

- [ ] Train classification model (strand → band)
- [ ] A/B testing against LLM scoring
- [ ] Gradual rollout

Phase 3.2: Evidence Ranking Model (Week 13-16)

- [ ] Train model to select best evidence clips
- [ ] Replace deterministic evidence generation
- [ ] Teacher feedback loop

Phase 3.3: Multimodal Fine-tuning (Week 17-24)

- [ ] Optional: Fine-tune vision model on labeled video
- [ ] Improved physical strand assessment
- [ ] Gesture and posture analysis

Phase 4: Scale & Enterprise (Months 7-12)

Page 292 of 349

Scalability:

- [ ] Read replicas for reporting queries
- [ ] Database sharding (by school)
- [ ] CDN for global video delivery
- [ ] Multi-region deployment

Enterprise Features:

- [ ] MIS integration (SIMS, Arbor, etc.)
- [ ] SAML/SSO authentication
- [ ] Custom rubric builder
- [ ] White-label options

Advanced AI:

- [ ] Group talk scoring (post-MVP feature)
- [ ] Real-time coaching feedback
- [ ] Predictive analytics

Page 293 of 349

Milestone Summary

Milestone	Date	Key Deliverables
MVP v0.1	Week 2	Solo mode, 3 modes, basic pipeline
MVP v0.2	Week 4	Stable, tested, documented
Beta	Week 8	Student portal, analytics
v1.0	Month 6	Custom models, enterprise ready
v2.0	Month 12	Group mode, advanced AI

Success Metrics (MVP)

Technical Metrics

- [ ] Video upload success rate > 95%

Page 294 of 349

- [ ] AI processing completion rate > 90%
- [ ] API response time p95 < 500ms
- [ ] Uptime > 99.5%
- [ ] Error rate < 1%

Quality Metrics

- [ ] Teacher edit rate < 30% (high AI accuracy)
- [ ] Evidence clip relevance (teacher satisfaction > 80%)
- [ ] Score confidence > 0.7 on 80% of assessments
- [ ] Transcription accuracy > 90%

User Metrics

- [ ] Time to complete assessment (upload → sign-off) < 10 min
- [ ] Teacher satisfaction score > 4/5
- [ ] Feature adoption rate > 80%

Page 295 of 349

Risk Mitigation

Risk	Probability	Impact	Mitigation
STT accuracy poor for child voices	Medium	High	Use Whisper fine-tuned on child speech, have fallback
LLM API rate limits	Medium	High	Implement queues, caching, request batching
Video storage costs	Medium	Medium	Implement lifecycle policies, compression
GDPR compliance issues	Low	Critical	Legal review, DPO consultation, encryption
Teacher adoption resistance	Medium	High	UX research, training materials, feedback loops
AWS region latency	Medium	Medium	CDN, multi-region planning

Team Structure (MVP)

Core Team (2-3 people)

Page 296 of 349

Full-Stack Developer (React + Django)

- Frontend implementation
- API development
- Database design
- DevOps support

ML/AI Engineer (Part-time)

- STT integration
- Feature extraction
- Prompt engineering
- Model evaluation

Product/UX (Part-time)

- User research
- UI/UX design
- Requirements refinement
- Testing coordination

Page 297 of 349

Development Workflow

Branch Strategy

```
main (production)
  ↑
staging (pre-production)
  ↑
feature/xyz (feature branches)
```

Pull Request Requirements

- [ ] Code review by 1 team member
- [ ] All tests passing

Page 298 of 349

- [ ] No security vulnerabilities (bandit, safety)
- [ ] Linting clean (black, flake8, eslint)
- [ ] Type checking (mypy, TypeScript)

Deployment Flow

1. Feature branch → PR
2. CI runs tests, linting, security checks
3. Code review
4. Merge to `staging`
5. Auto-deploy to staging environment
6. Manual QA on staging
7. Merge to `main`
8. Auto-deploy to production (blue/green)

Checkpoints & Reviews

Week 1 Review

Page 299 of 349

- [ ] Infrastructure ready
- [ ] Auth working
- [ ] Basic CRUD complete
- [ ] Video upload functional

Week 2 Review

- [ ] AI pipeline complete
- [ ] Reports generating
- [ ] Teacher UI functional
- [ ] End-to-end workflow tested

Pre-Launch Checklist

- [ ] Security audit passed
- [ ] Performance targets met
- [ ] Documentation complete

Page 300 of 349

- [ ] Training materials ready
- [ ] Support process defined
- [ ] Rollback plan documented

Deployment Strategy

Overview

This document outlines the CI/CD pipeline, environment management, and deployment procedures for Oracy AI.

CI/CD Pipeline

GitHub Actions Workflow

Page 301 of 349

```yaml
# .github/workflows/ci.yml

name: CI



on:

  push:

    branches: [main, staging]

  pull_request:

    branches: [main, staging]





jobs:

  test-backend:

    runs-on: ubuntu-latest

    services:

      postgres:

        image: postgres:15

        env:

          POSTGRESDB: oracytest

          POSTGRES_USER: postgres

          POSTGRES_PASSWORD: postgres

        options: >-

          --health-cmd pg_isready
```


Page 302 of 349

```
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
        ports:
```

- 5432:5432

```
      redis:
        image: redis:7
        ports:
```

- 6379:6379

```
steps:
```

- uses: actions/checkout@v4

Page 303 of 349

```
- name: Set up Python
      uses: actions/setup-python@v5
      with:
        python-version: '3.12'
- name: Install uv
      uses: astral-sh/setup-uv@v3
- name: Install dependencies
      run: |
        cd server
        uv pip install -e ".[dev]"
```


Page 304 of 349

```
- name: Run linting
      run: |
        cd server
        black --check .
        isort --check-only .
        flake8 .
        mypy .
- name: Run security checks
      run: |
        cd server
        bandit -r apps/ -f json -o bandit-report.json || true
        safety check
```


Page 305 of 349

```
  - name: Run tests




      env:

        DATABASEURL: postgres://postgres:postgres@localhost:5432/oracytest

        REDIS_URL: redis://localhost:6379/0

        SECRET_KEY: test-secret-key

      run: |

        cd server

        pytest --cov=apps --cov-report=xml --cov-report=html




  - name: Upload coverage




      uses: codecov/codecov-action@v3

      with:

        files: ./server/coverage.xml



test-frontend:

    runs-on: ubuntu-latest

    steps:
```


Page 306 of 349

```
- uses: actions/checkout@v4 ‌‌‌
```

```
- name: Run linting
      run: |
        cd client
        npm run lint
- name: Run type check
      run: |
        cd client
        npm run type-check
- name: Run unit tests
      run: |
```


Page 308 of 349

```
          cd client

          npm run test:unit -- --coverage




  • name: Build




        run: |

          cd client

          npm run build
```

Deployment Workflow

```
# .github/workflows/deploy.yml

name: Deploy



on:

  push:

    branches: [main, staging]
```


Page 309 of 349

```
env:
  AWS_REGION: ap-southeast-1
  ECR_REPOSITORY: oracy
  ECS_CLUSTER: oracy




jobs:
  build-and-push:
    runs-on: ubuntu-latest
    permissions:
      id-token: write
      contents: read




steps:



  - uses: actions/checkout@v4
```


Page 310 of 349

```
- name: Configure AWS credentials
    uses: aws-actions/configure-aws-credentials@v4
    with:
      role-to-assume: arn:aws:iam::ACCOUNT:role/GitHubActionsDeployRole
      aws-region: ${{ env.AWS_REGION }}
- name: Login to Amazon ECR
    id: login-ecr
    uses: aws-actions/amazon-ecr-login@v2
- name: Build, tag, and push backend image
    env:
      ECR_REGISTRY: ${{ steps.login-ecr.outputs.registry }}
      IMAGE_TAG: ${{ github.sha }}
```


Page 311 of 349

```
        run: |
          docker build -t $ECRREGISTRY/$ECRREPOSITORY-django:$IMAGE_TAG ./server
          docker push $ECRREGISTRY/$ECRREPOSITORY-django:$IMAGE_TAG
```

- name: Build, tag, and push frontend image

```
        env:
          ECR_REGISTRY: ${{ steps.login-ecr.outputs.registry }}
          IMAGE_TAG: ${{ github.sha }}
        run: |
          docker build -t $ECRREGISTRY/$ECRREPOSITORY-react:$IMAGE_TAG ./client
          docker push $ECRREGISTRY/$ECRREPOSITORY-react:$IMAGE_TAG
```

```
deploy-backend:
    needs: build-and-push
    runs-on: ubuntu-latest
    steps:
```

- uses: actions/checkout@v4

Page 312 of 349

```
  - name: Configure AWS credentials
      uses: aws-actions/configure-aws-credentials@v4
      with:
        role-to-assume: arn:aws:iam::ACCOUNT:role/GitHubActionsDeployRole
        aws-region: ${{ env.AWS_REGION }}
  - name: Update ECS service
      run: |
        aws ecs update-service \
          --cluster $ECS_CLUSTER \
          --service django-api \
          --force-new-deployment
```


Page 313 of 349

```
  - name: Wait for deployment
        run: |
          aws ecs wait services-stable \
            --cluster $ECS_CLUSTER \
            --services django-api

deploy-frontend:
    needs: build-and-push
    runs-on: ubuntu-latest
    steps:

  - uses: actions/checkout@v4

  - name: Configure AWS credentials
```


Page 314 of 349

```
      uses: aws-actions/configure-aws-credentials@v4
      with:
        role-to-assume: arn:aws:iam::ACCOUNT:role/GitHubActionsDeployRole
        aws-region: ${{ env.AWS_REGION }}




  - name: Deploy to S3



      run: |
        aws s3 sync ./client/dist s3://oracy-frontend-${{ github.ref_name }} \
          --delete \
          --cache-control "max-age=31536000,immutable"



  - name: Invalidate CloudFront



      run: |
        aws cloudfront create-invalidation \
          --distribution-id ${{ secrets.CLOUDFRONTDISTRIBUTIONID }} \
          --paths "/*"
```


Page 315 of 349

Environment Management

Environment Structure

```
┌──────────────────────────────────────────────────────────────────┐
│                        PRODUCTION                                 │
│  • oracy.ai                                                       │
│  • AWS Account: oracy-prod                                        │
│  • RDS: db.r6g.xlarge (Multi-AZ)                                  │
│  • ECS: 4-10 tasks                                                │
│  • S3: oracy-prod-media                                           │
│  • Backup: 7 days                                                 │
└──────────────────────────────────────────────────────────────────┘
                                ▲
                                │ Promote after QA
┌──────────────────────────────────────────────────────────────────┐
│                        STAGING                                    │
```


Page 316 of 349

```
│  • staging.oracy.ai                                          │
│  • AWS Account: oracy-staging                                │
│  • RDS: db.t3.medium (Single AZ)                             │
│  • ECS: 2 tasks                                              │
│  • S3: oracy-staging-media                                   │
│  • Data: Anonymized production snapshot                      │
└──────────────────────────────────────────────────────────────┘

                              ▲
                              │ Auto-deploy on merge
┌──────────────────────────────────────────────────────────────┐
│                      DEVELOPMENT                             │
│  • localhost / Docker Compose                                │
│  • Local PostgreSQL + Redis                                  │
│  • Mock external services (optional)                         │
└──────────────────────────────────────────────────────────────┘
```

Environment Variables per Environment

```
# .env.production

DJANGOSETTINGSMODULE=config.settings.production
```


Page 317 of 349

```
DEBUG=False

SECRET_KEY=

DATABASE_URL=postgres://user:pass@prod-rds:5432/oracy

REDIS_URL=redis://prod-redis:6379/0

AWSSTORAGEBUCKET_NAME=oracy-prod-media

OPENAIAPIKEY=

SENTRY_DSN=
```

```
# .env.staging

DJANGOSETTINGSMODULE=config.settings.staging

DEBUG=False

SECRET_KEY=

DATABASE_URL=postgres://user:pass@staging-rds:5432/oracy

REDIS_URL=redis://staging-redis:6379/0

AWSSTORAGEBUCKET_NAME=oracy-staging-media

OPENAIAPIKEY=

SENTRY_DSN=
```


Page 318 of 349

Blue/Green Deployment

ECS Blue/Green with CodeDeploy

```
# Terraform: ECS Blue/Green deployment

resource "awscodedeployapp" "oracy" {

  name             = "oracy-django"

  compute_platform = "ECS"

}




resource "awscodedeploydeployment_group" "oracy" {

  appname                = awscodedeploy_app.oracy.name

  deploymentgroupname   = "oracy-django-dg"

  servicerolearn        = awsiamrole.codedeploy.arn

  deploymentconfigname = "CodeDeployDefault.ECSAllAtOnce"





bluegreendeployment_config {

    deploymentreadyoption {

      actionontimeout = "CONTINUE_DEPLOYMENT"

    }
```


Page 319 of 349

```
terminateblueinstancesondeployment_success {
      action                          = "TERMINATE"
      terminationwaittimeinminutes = 30
    }
  }


ecs_service {
    clustername = awsecs_cluster.oracy.name
    servicename = awsecs_service.django.name
  }


deployment_style {
    deploymentoption = "WITHTRAFFIC_CONTROL"
    deploymenttype   = "BLUEGREEN"
  }
}
```


Page 320 of 349

Deployment Process

1. **Build** - Create Docker images with commit SHA tag
2. **Push** - Upload to ECR
3. **Register** - Register new task definition
4. **Deploy** - CodeDeploy creates green environment
5. **Health Check** - Verify green environment health
6. **Traffic Shift** - Gradually shift traffic (10% → 50% → 100%)
7. **Monitor** - Watch error rates for 30 minutes
8. **Cleanup** - Terminate blue environment on success

Rollback Procedure

```
# Manual rollback to previous version

aws ecs update-service \

  --cluster oracy \

  --service django-api \

  --task-definition oracy-django:PREVIOUS_REVISION \

  --force-new-deployment



# Or via CodeDeploy



aws deploy rollback-deployment \
```


Page 321 of 349

```
  --deployment-id d-ABCD1234
```

Database Migrations

Zero-Downtime Migration Strategy

```
# Migration safety rules

"""

1. Add new columns as nullable (or with defaults)

2. Create indexes CONCURRENTLY

3. Backfill data in batches

4. Make column non-nullable in separate migration

5. Remove old columns in later release

"""



# Example safe migration
```


Page 322 of 349

```
class Migration(migrations.Migration):

    atomic = False  # Allow concurrent index creation



    operations = [

        # Step 1: Add new column as nullable

        migrations.AddField(

            model_name='assessment',

            name='new_field',

            field=models.CharField(max_length=100, null=True),

        ),



        # Step 2: Create index concurrently

        migrations.RunSQL(

            sql='CREATE INDEX CONCURRENTLY idxassessmentsnew ON assessments(new_field)

            reversesql='DROP INDEX idxassessments_new;'

        ),

    ]
```

Migration Deployment Process

Page 323 of 349

```
# 1. Run migrations BEFORE deploying new code
```

(ensure backward compatibility)

```
python manage.py migrate --database=production
```


Page 324 of 349

2. Deploy new code

Page 325 of 349

3. Verify migrations applied

```
python manage.py showmigrations
```


Page 326 of 349

```
4. If rollback needed, rollback code first,


python manage.py migrate appname PREVIOUSMIGRATION
```

Monitoring & Alerting

CloudWatch Dashboards

```
{
  "widgets": [
    {
      "type": "metric",
      "properties": {
        "metrics": [
          ["AWS/ECS", "CPUUtilization", "ServiceName", "django-api", "ClusterName", "o
          ["...", "celery-worker", ".", "."]
```


Page 327 of 349

```
        ],
        "period": 300,
        "stat": "Average",
        "region": "ap-southeast-1",
        "title": "ECS CPU Utilization"
      }
    },
    {
      "type": "metric",
      "properties": {
        "metrics": [
          ["AWS/RDS", "CPUUtilization", "DBInstanceIdentifier", "oracy-postgres"],
          ["...", "DatabaseConnections", ".", "."]
        ],
        "period": 300,
        "stat": "Average",
        "region": "ap-southeast-1",
        "title": "RDS Metrics"
      }
    }
  ]
}
```


Page 328 of 349

Alerting Rules

```
# CloudWatch Alarms

HighCPUAlarm:

  Type: AWS::CloudWatch::Alarm

  Properties:

    AlarmName: "Oracy-HighCPU"

    MetricName: CPUUtilization

    Namespace: AWS/ECS

    Statistic: Average

    Period: 300

    EvaluationPeriods: 2

    Threshold: 80

    ComparisonOperator: GreaterThanThreshold

    AlarmActions:
```

- `arn:aws:sns:ap-southeast-1:ACCOUNT:oracy-alerts`

Page 329 of 349

```
HighErrorRateAlarm:

  Type: AWS::CloudWatch::Alarm

  Properties:

    AlarmName: "Oracy-HighErrorRate"

    MetricName: 5xxErrorRate

    Namespace: AWS/ApplicationELB

    Statistic: Average

    Period: 60

    EvaluationPeriods: 2

    Threshold: 1

    ComparisonOperator: GreaterThanThreshold

    AlarmActions:
```

- arn:aws:sns:ap-southeast-1:ACCOUNT:oracy-alerts

```
DatabaseConnectionsAlarm:

  Type: AWS::CloudWatch::Alarm

  Properties:

    AlarmName: "Oracy-HighDBConnections"

    MetricName: DatabaseConnections
```


Page 330 of 349

```
    Namespace: AWS/RDS

    Statistic: Average

    Period: 300

    EvaluationPeriods: 2

    Threshold: 150

    ComparisonOperator: GreaterThanThreshold
```

Error Tracking (Sentry)

```
# config/settings/production.py

import sentry_sdk

from sentry_sdk.integrations.django import DjangoIntegration

from sentry_sdk.integrations.celery import CeleryIntegration



sentry_sdk.init(

    dsn=os.getenv('SENTRY_DSN'),

    integrations=[

        DjangoIntegration(),

        CeleryIntegration(),

    ],
```


Page 331 of 349

```
    tracessamplerate=0.1,
    profilessamplerate=0.1,
    environment="production",
    release=os.getenv('GITCOMMITSHA'),
)
```

Disaster Recovery

Backup Strategy

Component	Backup Method	Frequency	Retention
RDS	Automated snapshots + manual	Daily	7 days
RDS	Cross-region snapshot	Weekly	30 days
S3	Versioning + Cross-region replication	Continuous	Versioned
Application config	Terraform state + Secrets Manager	On change	Versioned

Recovery Procedures

Page 332 of 349

RDS Point-in-Time Recovery

```
# Restore to specific point in time

aws rds restore-db-instance-to-point-in-time \

  --source-db-instanceIdentifier oracy-postgres \

  --target-db-instance-identifier oracy-postgres-recovery \

  --restore-time 2026-03-01T12:00:00Z
```

Cross-Region Failover

```
# Promote read replica in secondary region

aws rds promote-read-replica \

  --db-instance-identifier oracy-postgres-replica
```

Update application to use new endpoint

Page 333 of 349

```
(via environment variable or parameter stor
```

RTO/RPO Targets

Scenario	RTO (Recovery Time)	RPO (Data Loss)
Single AZ failure	5 minutes (Multi-AZ)	0
Database corruption	30 minutes	5 minutes (PITR)
Region failure	2 hours	1 hour (async replication)
Complete loss	4 hours	1 hour

Local Development

Docker Compose Setup

```
# docker-compose.yml

version: '3.8'
```


Page 334 of 349

```
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_DB: oracy
      POSTGRES_USER: oracy
      POSTGRES_PASSWORD: oracy
    volumes:
```

- postgres_data:/var/lib/postgresql/data

```
    ports:
```

- "5432:5432"

```
redis:
    image: redis:7
```


Page 335 of 349

```
    ports:
```

- "6379:6379"

```
backend:
    build:
      context: ./server
      dockerfile: Dockerfile.dev
    environment:
      DATABASE_URL: postgres://oracy:oracy@db:5432/oracy
      REDIS_URL: redis://redis:6379/0
      DEBUG: "true"
    volumes:
```

- ./server:/app

```
    ports:
```

- "8000:8000"

Page 336 of 349

```
    depends_on:
```

- db
- redis

```
frontend:
    build:
      context: ./client
      dockerfile: Dockerfile.dev
    environment:
      VITEAPIURL: http://localhost:8000/api
    volumes:
```

- ./client:/app
- /app/node_modules

Page 337 of 349

```
    ports:
      - "5173:5173"
    depends_on:
      - backend

celery:
    build:
      context: ./server
      dockerfile: Dockerfile.dev
    command: celery -A config worker -l info
    environment:
      DATABASE_URL: postgres://oracy:oracy@db:5432/oracy
      REDIS_URL: redis://redis:6379/0
    volumes:
```


Page 338 of 349

```
    - ./server:/app
    depends_on:
    - db
    - redis
volumes:
  postgres_data:
```

Local Development Commands

```
# Start all services
docker-compose up -d
```


Page 339 of 349

Run migrations

```
docker-compose exec backend python manage.py migrate
```


Page 340 of 349

Create superuser

```
docker-compose exec backend python manage.py createsuperuser
```


Page 341 of 349

Run tests

```
docker-compose exec backend pytest

docker-compose exec frontend npm run test
```


Page 342 of 349

View logs

```
docker-compose logs -f backend

docker-compose logs -f celery
```


Page 343 of 349

Reset database

```
docker-compose down -v

docker-compose up -d db
```

Release Checklist

Pre-Deployment

- [ ] All tests passing
- [ ] Security scan clean (bandit, safety, npm audit)
- [ ] Database migrations reviewed
- [ ] Feature flags configured
- [ ] Monitoring dashboards verified
- [ ] Rollback plan documented

Page 344 of 349

Deployment

- [ ] Deploy to staging first
- [ ] Run smoke tests on staging
- [ ] Notify team of production deployment
- [ ] Deploy to production (blue/green)
- [ ] Verify health checks pass
- [ ] Monitor error rates for 30 minutes

Post-Deployment

- [ ] Verify critical user journeys
- [ ] Check Sentry for new errors
- [ ] Monitor CloudWatch metrics
- [ ] Update runbooks/documentation

Page 345 of 349

- [ ] Announce deployment complete

Rollback Criteria

- Error rate > 1% for 5 minutes
- Any 500 errors on critical paths
- Performance degradation > 50%
- Data integrity issues
- Security incident

Secrets Management

AWS Secrets Manager Structure

Page 346 of 349

```
oracy/
├── production/
│   ├── django-secret-key
│   ├── database-credentials
│   ├── openai-api-key
│   └── jwt-signing-key
├── staging/
│   ├── django-secret-key
│   ├── database-credentials
│   ├── openai-api-key
│   └── jwt-signing-key
└── development/
    └── (local .env files)
```

Secret Rotation Schedule

Secret	Rotation Frequency	Process
Database password	90 days	Update RDS, restart services
API keys	180 days	Generate new, update secrets, restart
JWT signing key	On demand	Invalidate all sessions
Django secret	On demand	Session flush required

Page 347 of 349

Cost Monitoring

AWS Cost Alarms

```
BudgetAlarm:

  Type: AWS::Budgets::Budget

  Properties:

    Budget:

      BudgetName: "Oracy-Monthly-Budget"

      BudgetLimit:

        Amount: 2000

        Unit: USD

      TimeUnit: MONTHLY

      BudgetType: COST

    NotificationsWithSubscribers:



  • Notification:
```


Page 348 of 349

```
          NotificationType: ACTUAL

          ComparisonOperator: GREATER_THAN

          Threshold: 80

        Subscribers:




  • SubscriptionType: SNS




            Address: arn:aws:sns:ap-southeast-1:ACCOUNT:oracy-alerts
```

Cost Optimization Measures

- Use Spot instances for Celery workers (dev/staging)
- S3 Intelligent Tiering for old videos
- RDS Reserved Instances for production
- CloudFront caching to reduce origin requests
- Lambda for lightweight processing (optional)

Page 349 of 349